eCAT User Guide

Python interface guide | eCAT 0.1.0b8 | 25 October 2026

Contents

Quick orientation

Installation

eCAT app

Five-minute usage path

Function and class index

Loading data

Plotting and grouping

CV analysis

CA/CP basics and limitations

Advanced analysis

Simulation namespace

Troubleshooting

Scope and limitations

Detailed Function Reference

Extracting, Organizing, & Saving Data

Objects and result ownership

Plotting

Individual CV Data Analysis

CA CP and DPV reference

Complex Data Analysis

Simulation and fitting

Defaults and Option Discovery

Function finder

Select a name to jump to its reference. Word's Navigation pane follows the same heading hierarchy.

Data collection and naming

open_app

get_data

echem.from_file

parse_file

filter

sort

group

sort_and_group

group_summary

save_data

get_data_from_excel

echem.metadata

echem.xy

Object settings and unit conversions

show

show_objects

AnalysisResult.to_dataframe

echem.plot

multiplot

multimultiplot

multi_scatterplot

animate

plotting_style

echem.spectrum

Peak and wave call conventions

cv.peak_potential

cv.peak_current

cv.peak_width

cv.half_peak_potential

cv.peak_info

cv.half_wave_potential

cv.wave_info

trim

cv.filter

cv.analysis_segment_data

cv.current_at_potential

cv.plot_program

ca.current_at_time

ca.average_current

ca.rate_at_time

ca.average_rate

ca.charge

ca.time_at_charge

cp.get_cycles

cp.cycle_info

cp.plot_cycles

cp.cycling_plot

dpv.peak_potential

dpv.fit_overlapping_peaks

fowa

fit_peak_current

fit_peak_potential

fit_model

fit_rate

sevcik_analysis

reversibility_analysis

surface_coverage_analysis

plateau_current

trumpet_analysis

nicholson_analysis

scale_current

normalize_current

normalize

tafel_analysis

Release scope

Simulation objects and ownership

simulation.compile_mechanism

simulation.cv_program

simulation.cv_data

simulation.simulate_cv

simulation.fit_cv

simulation.fit_cvs

Simulation parameter units and preparation

Examples

describe_options

get_defaults

set_defaults

reset_defaults

load_defaults

Typed options and error classes

Quick orientation

eCAT loads, inspects, plots, filters, analyzes, and exports electrochemistry data. Start with get_data() for a folder or echem.from_file() for one file, inspect the resulting objects, and choose analyses that match the experiment. This guide describes the 0.1.0b8 release candidate. Check e.__version__ and describe_options() in the environment running your notebook. The optional simulation API uses standard ElectroKitty; experimental backend and adaptive-grid work is excluded.

Installation

Install eCAT in a clean Python environment, then import it as e. If Matplotlib reports a cache-directory warning on first import, set MPLCONFIGDIR to a writable folder or continue after the cache is built.

eCAT app

Install the optional app dependencies with python -m pip install "ecat-electrochemistry[app]". For an editable source checkout use python -m pip install -e ".[app]". The terminal launcher is:

```
ecat-app
```

Notebook users can launch the same native app from Python:

```
import ecat as e

e.open_app()
```

Use ecat-app --browser or e.open_app(browser=True) if you prefer a browser tab. Use e.open_app(inline=True) to display browser mode in a notebook iframe when the notebook environment supports it. Browser mode opens at http://127.0.0.1:8050 by default and automatically uses the next available port if 8050 is busy; native-window mode uses a private available port by default.

Five-minute usage path

```
import ecat as e

objects = e.get_data({"folder path": "path/to/exports", "print": False})
e.show_objects(objects)
# After confirming a CV was loaded:
cvs = e.filter(objects, {"type": "CV"}, {"print": False})
if cvs:
    cvs[0].plot()
    metadata = cvs[0].metadata()
```

Function and class index

Core objects: echem, cv, ca, cp, dpv.

Loading: echem.from_file, parse_file, get_data, get_data_from_excel.

Inspection: show, metadata, x, y, xy, show_objects, and filter(objects) for available metadata values.

Plotting: plot, animate, multiplot, multimultiplot, multi_scatterplot, plotting_style. Scale bars are provided through the scale bar plot option.

Filtering and grouping: filter, sort, group, sort_and_group, group_summary, filter (inspection).

CV analysis: peak_potential, peak_current, peak_width, half_peak_potential, half_wave_potential, current_at_potential, normalize, normalize_current, scale_current.

Advanced analysis: fowa, sevcik_analysis, trumpet_analysis, nicholson_analysis, tafel_analysis, fit_model, fit_rate, plateau_current, fit_peak_potential, fit_peak_current.

Export and defaults: save_data, describe_options, get_defaults, set_defaults, reset_defaults, load_defaults.

Simulation: optional workflows under e.simulation; see the release API and numerical limitations in the simulation chapter.

Loading data

Each imported object retains obj.parse_result. Its ParseResult fields include data, units, technique, software, metadata, raw_metadata, warnings, source, and parser. Use e.parse_file(filepath, options) for this contract directly; use e.echem.from_file(filepath, options) for an analysis object.

The ParseResult-first text paths cover tested CH, BASI, EC-Lab-style, limited NOVA ASCII, and generic numeric/header imports. Ambiguous generic tables containing time, potential and current remain generic echem objects unless technique metadata resolves the experiment. IviumSoft and DPV formats outside the tested fixtures remain limited.

Use get_data() for folders of supported text files and get_data_from_excel() for eCAT Excel workbooks or curated spreadsheet CV tables. EC-Lab ASCII .mpt is supported; binary .mpr is rejected with export guidance. Folder imports skip unsupported files and return the successfully loaded objects, or [] when none load.

Data collection and naming

Recommended names make loading, filtering, grouping, plotting, and simulation fitting much smoother because they give eCAT reliable metadata even when instrument exports are sparse.

- Put technique, solvent, gas/atmosphere, scan rate or window, compounds, concentrations, and run/replicate identifiers in the file or folder name when those values may not survive the instrument export.
- Keep each concentration adjacent to its compound name when possible, such as 100mMPhOH, 1mMCo, 3mMFc, or 5%CO_2.
- Use explicit units: M, mM, uM, µM, nM, %, equiv, and x are recognized. Avoid bare lowercase m for molarity; write 100mM, not 100m.
- Choose species names that match later analysis or simulation names when practical. If experimental names differ from simulation species, record the mapping in the notebook with options such as concentration mapping.
- Good compact examples: 100mVs_CO2_MeCN_1mMFe-tpyPY2Me_100mMPhOH_run01 or CV_MeCN_Ar_0.1MTBAPF6_3mMFc_1mMFe_100mVs.

Plotting and grouping

Use obj.plot() for one object, animate() or obj.animate() for notebook-friendly playback, and multiplot() for overlays. animate() auto-displays in notebooks when 'plot' is True, suppresses the empty static placeholder figure, and returns an AnimationResult for reuse or saving. Use plotting_style("notebook"), plotting_style("publication"), or plotting_style(False) to control eCAT plotting defaults.

- Use filter(), sort(), group(), and sort_and_group() to organize loaded objects before plotting. The package default plot convention is IUPAC; for CVs, use {'plot convention': 'US'} when you want the historical US axis orientation instead.

Use the plot option symbol labels to switch axis labels from descriptive text to compact electrochemistry symbols. With symbol labels enabled, potential, current, current density, time, and charge are labeled E, i, j, t, and Q. The Savéant plotting style enables symbol labels automatically; notebook and publication styles keep descriptive labels unless symbol labels is set to True. Normalized axes such as i/i_p^0 remain symbolic.

CV analysis

CV-specific analysis includes peak potentials, peak currents, half-wave estimates, current normalization, dimensionless normalization, and current scaling. Top-level normalize() is CV-only and returns copies; cv.normalize() mutates one CV object.

CA/CP basics and limitations

CA supports time-resolved current, charge, electron-flow and conditional molecular-rate analysis. CP supports cycle segmentation and capacity/efficiency summaries. DPV supports peak detection and a two-Gaussian overlap fit. The dedicated CA, CP and DPV chapter below gives arguments, return types, units and current limitations. Imported metadata and actual signal arrays remain distinct; inspect obj.metadata() and obj.parse_result.warnings before analysis.

CA objects also expose current_at_time(), average_current(), rate_at_time(), and average_rate(). These helpers return dict-like ChronoAnalysisResult objects with a Metric/Value table and print the relevant interpolation, averaging, or rate equations by default. Rate helpers report electron flow from i/F and, when electron count, Faradaic efficiency, and catalyst amount are supplied, molecular rate and TOF.

Advanced analysis

Advanced workflows such as fowa, Sevcik-style analysis, trumpet analysis, Nicholson analysis, Tafel-style analysis, and fit helpers should be used with attention to assumptions, fit ranges, and warnings.

Advanced analysis return contract: complex analyses return AnalysisResult-style objects. Use .table for the primary table, .summary for workflow metadata, .fits or .fit_table where fitting applies, .axes for plots, .units for result units, and .warnings or .diagnostics for extra detail. Use result.to_csv(...) to export the primary table as CSV, or result.to_excel(...) to write an Excel workbook with table, summary, fit_table, fits, warnings, units, and diagnostics sheets when present.

Simulation namespace

Simulation: optional workflows under e.simulation; see the release API and numerical limitations in the simulation chapter.

Troubleshooting

When something fails, record the package version, function call, options, file type, expected behavior, actual behavior, and traceback. Common issues include unsupported file formats, unit confusion, reference-shift setup, normalization parameters, missing peaks, and parser failures.

For parser issues, inspect obj.parse_result.warnings and obj.parse_result.raw_metadata. Warnings are nonfatal diagnostics that often indicate missing or inferred metadata such as scan rate, timestamp, step structure, or technique. Use e.parse_file(path) to inspect the parser contract directly before object promotion.

Scope and limitations

The priority is trustworthy core workflows, clear limitations, and useful feedback from real lab data. Optional features should not block installation, loading, plotting, core CV analysis, export, and documentation.

Parser scope note: binary vendor files remain unsupported. Generic text import is a fallback, and IviumSoft or DPV text workflows should be validated with representative files before quantitative interpretation.

Detailed Function Reference

Use import ecat as e in notebooks and scripts. Check e.__version__ in the active kernel before comparing options with this source-checkout guide.

```
# Run in the notebook kernel you will use
%pip install "ecat-electrochemistry[app]"

import ecat as e
print(e.__version__)
```

Options model Public functions still accept simple dictionaries. Internally, eCAT validates option names, values, choices, and routing before analysis starts. Use e.describe_options('function_name') to inspect the full schema from a notebook.

Common option helpers

Task	Notebook command	Notes
Inspect options	e.describe_options('multiplot')	Shows defaults, choices, and descriptions.
Set one default	e.set_defaults('fowa', {'fit basis': 'y'})	Section-specific defaults override package defaults.
Set a global default	e.set_defaults('print', False)	Applies to option models that support print.
Reset defaults	e.reset_defaults()	Restores package defaults for the session.

open_app

```
e.open_app(host='127.0.0.1', port=0, *, mode='local', browser=False, open_browser=True,
inline=False, height=900, width=1400, title='eCAT Workbench', allow_code_execution=False,
quiet=False)
```

Launch the optional local Workbench and return its URL.

Parameters

Parameter	Meaning and default
host, port	127.0.0.1 and 0. Browser mode starts searching at 8050; native mode requests a free port.
mode	"local" by default; forwarded as ECAT_APP_MODE.
browser, inline	False by default. Either selects browser-server mode; inline also requests an iframe.
open_browser	True; browser mode opens a tab. False avoids that side effect.
height, width, title	900 px, 1400 px and "eCAT Workbench"; native window sizing/title. height also controls iframe height.
allow_code_execution	False. Opt-in local app code execution; disabled in remote mode.
quiet	False; suppresses launcher URL printing when True.

Returns

str: local URL. Native mode returns after the window closes; browser mode runs a daemon server thread.

Limitations and behavior

- Install the app extra in the notebook kernel environment. Do not infer standalone macOS signing/notarization from this Python launcher.

Example

```
url = e.open_app(browser=True)
```

Extracting, Organizing, & Saving Data

get_data

```
e.get_data(options=None)
```

Reads electrochemical text files, creates echem/cv/cp/ca/dpv objects, sorts them, and attaches reference-potential metadata when requested.

Parameters

Parameter	Meaning
options (dict or ImportOptions)	import, parser, metadata, sorting, and reference-shift controls.

Key options

Option	Default / type	Purpose and choices
folder path	'.' str	Folder path searched for electrochemical data files.
recursive search	True bool	Recursively search subfolders during import.
sort keys	['subfolder', 'timestamp'] object	Sort keys used to order imported objects after parsing and before reference assignment.
software	None str or None	Instrument software/parser. If omitted, import tries to infer the parser from file contents and extension.
reference mode	'auto' str	'auto' searches imported files with reference keywords; 'keyword', 'file', and 'manual' use the corresponding explicit reference source. Automatic peak pairing requires adjacent, opposite-direction CV segments. Choices: auto, manual, keyword, file, none.
reference keyword	None str or None	Single keyword used by reference mode 'keyword' to choose reference files.
reference file	None str or None	Explicit reference file used by reference mode 'file'; no file matching is inferred.
reference guess	'auto' float or str or None	'auto' locates the reference wave automatically and raises when multiple adjacent-segment couples are indistinguishable; a numeric value ranks physical pairs near that potential.
electrode diameter	0 float	Electrode diameter in cm used to compute electrode area when electrode area is not provided.

Option	Default / type	Purpose and choices
electrode area	0 float	Electrode area in cm^2 assigned to imported objects for later current-density plotting, normalization, and kinetic analysis.
invert current	False bool	Multiply current by -1.
name alterations	None object or None	Filename text replacements applied during import.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
troubleshoot	False bool	Print import diagnostics plus reference segment/candidate tables and display the segment-colored reference-pair plot.
reference map	None dict or None	Explicit target-to-reference object index mapping; overrides automatic reference assignment for those objects.

Behavior notes

- 'reference map' uses the sorted/displayed object indices and overrides automatic self/folder reference assignment for the mapped targets.
- Default folder import order keeps subfolders together, then orders objects within each subfolder by acquisition timestamp when instrument timestamps are available. Timestamp sorting falls back to filesystem creation and modification times, with folder-relative path ordering used for stable discovery.
- Generic fallback objects parse gas and solvent from filenames. CH-style CPE/CA, CV, and DPV imports keep IR compensation metadata for sorting, grouping, and table/stat display when those fields are requested. Single-object show() displays IR compensated and uncompensated resistance values with ohm in the value column. For single-file loading, e.echem.from_file(path, {'print': True}) displays the loaded object with e.show().
- When 'reference mode' is 'file', 'reference file' may be absolute or relative to 'folder path'. eCAT loads that file, locates the reference couple using the same reference-guess and shift-window options, and applies the resulting potential shift to imported CVs with reference mode 'file'.
- Automatic reference-wave detection pairs extrema only from adjacent, opposite-direction CV segments. E_{pa} is assigned from the increasing-potential sweep and E_{pc} from the decreasing-potential sweep, independent of current sign. A numeric 'reference guess' ranks valid physical pairs near that potential; 'auto' raises a clear ambiguity error when multiple couples are indistinguishable.
- Set 'troubleshoot': True for segment/candidate tables and a segment-colored plot. The selected E_{pa}, E_{pc}, ΔE_p, segments, directions, extrema, prominences, and selection mode are retained in obj.reference_pair_details and ParseResult metadata.

EC-Lab GCPL exports are imported as chronopotentiometry (cp) objects when the technique line is Galvanostatic Cycling with Potential Limitation. Numeric time columns and absolute timestamp columns are converted to elapsed seconds, Ewe is stored as Potential in V, and programmed Is step currents are captured as anodic_current and cathodic_current when present.

- If an individual .txt file cannot be converted, get_data() prints a warning with the filename and error, skips that file, and returns the successfully imported objects. If no files can be converted, it prints a warning and returns an empty list [].

Returns

list[echem]. If no files convert successfully, returns an empty list [] after printing a warning.

Example

```
objects = e.get_data(
    {
        "folder path": r"C:/data/CV_runs",
        "recursive search": True,
        "reference mode": "keyword",
        "reference keyword": "Fc",
        "reference map": {45: 54},
    }
)
```

- Filename metadata recognizes mole-fraction tokens with the unit x, such as 0.8xD2O, 0xD2O, and 1xD2O. Endpoint mole fractions must be explicit; missing x tokens are not inferred as 0 or 1. User-facing labels display mole fractions as χ(species), for example χ(D2O).

Import output shows one totals summary. Recursive imports with subfolders also show a DataFrame for every visited folder, including empty folders, with candidate, loaded, failed, unsupported and reference-use counts. A single folder shows totals without a redundant folder table. Relative subfolder paths are displayed as stored, using forward slashes. Unsupported-extension and binary filename previews are capped at five entries. Reference summaries distinguish explicit files, keyword/folder assignment and fallback; a reference count can overlap loaded-file counts. print=False suppresses reporting.

Filename parsing and recommended names

eCAT uses filename parsing as a practical metadata fallback when a file does not provide gas, solvent, compound/concentration, or scan-rate metadata clearly.

- Built-in filename parsing looks for gas, solvent, compounds, concentrations, and scan rate.
- Recognized concentration units in filenames include M, mM, uM, µM, nM, L, %, equiv, and x.
- Scan-rate filename forms include 100mVs, 100 mV/s, 0.1Vs, 250mV_s, 0p1V_s and 500uV/s. Decimal marks ., comma and p are supported inside numeric tokens; comma is a decimal mark, not a thousands separator.
- Recognized pressure units for metadata and display include Pa, bar, atm, Torr, mmHg, and psi. SI prefixes are supported where they are natural for pressure, such as kPa, MPa, and mbar. eCAT preserves pressure units by default and converts between pressure units only when a target/display unit is requested explicitly.
- Recommended filename style is explicit and compact, for example 100mVs_CO2_MeCN_1mMFe-tpyPY2Me_100mMPhOH_run01 or CV_MeCN_Ar_0.1MTBAPF6_3mMFc_1mMFe_100mVs.
- Keep species and concentration together, using underscores to separate fields. Spaces within compound names are retained; for example 0p5mM_ferrocene carboxylic acid_10mM_TBAPF6_0p1Vs. DMA and DMAc are recognized solvent tokens and retain their entered spelling.
- Bare lowercase m is intentionally not treated as molar because it is too ambiguous. If the built-in parser is not enough, use the compounds option, a custom parser, or parser settings.
- Scan-rate precedence is instrument header, then a reliable potential/time waveform estimate, then filename fallback. Control Potential is preferred for estimating the programmed rate when present. Conflicts are flagged and original candidates remain in parser provenance; measured potentials and currents are not replaced.
- Set parser settings with prefer file metadata=False only for an intentional custom metadata override. EC-Lab cycle numbers and eCAT segments are different: a segment is a monotonic sweep; inspect the resolved segment boundaries instead of doubling a cycle label blindly.

EC-Lab export requirements

A CV export must contain measured potential (Ewe/V or <Ewe>/V) and current (I/mA or <I>/mA); include time/s when available. Files containing only time, current range, power and resistance cannot reconstruct a CV reliably. The parser now names missing channels and asks for a complete export from the original measurement.

echem.from_file

```
echem.from_file(filepath, options=None)
```

Load one supported file into the detected echem subclass.

Parameters

Parameter	Meaning and default
filepath	File path; required.
options	ImportOptions or dictionary; None uses defaults.

Returns

An echem, cv, ca, cp or dpv object with data, units, metadata and parse_result.

Limitations and behavior

- Binary .mpr input raises UnsupportedFileFormatError. Text fallback does not prove a technique assignment.

Full options: e.describe_options("get_data").

Example

```
obj = e.echem.from_file("path/to/trace.mpt", {"print": False})
metadata = obj.metadata()
```

Selected options

Option	Default	Meaning
allow self reference	True	Allow a CV to be considered as its own reference candidate during automatic reference shifting.
peak prominence	None	Minimum peak prominence for reference extrema on each segment; None estimates a separate threshold from each segment's search region.
reference file	None	Explicit reference file used by reference mode 'file'; no file matching is inferred.
reference guess	'auto'	'auto' locates the reference wave automatically and raises when multiple adjacent-segment couples are indistinguishable; a numeric value ranks physical pairs near that potential.
reference keyword	None	Single keyword used by reference mode 'keyword' to choose reference files.
reference keywords	['Fc', 'ferrocene', 'decamethylferrocene', 'DmFc', 'cobaltocene', 'CoCp2']	Reference keywords tried by reference mode 'auto'; the first successful keyword defines the shift source.
reference label	'Fc/Fc+'	Axis label used after reference shifting.
reference map	None	Explicit target-to-reference object index mapping; overrides automatic reference assignment for those objects.
reference max delta ep	0.2	Largest allowed Epa - Epc separation in V for an adjacent-segment reference pair.

Option	Default	Meaning
reference mode	'auto'	'auto' searches imported files with reference keywords; 'keyword', 'file', and 'manual' use the corresponding explicit reference source. Automatic peak pairing requires adjacent, opposite-direction CV segments. Choices: auto, manual, keyword, file, none.
reference offset	None	Manual reference potential offset.
reference smooth	True	Smooth each complete CV segment independently before reference extrema are detected; smoothing never crosses a switching vertex.

Complete registered options: e.describe_options("get_data").

EC-Lab export requirements

A CV export must contain measured potential (Ewe/V or <Ewe>/V) and current (I/mA or <I>/mA); include time/s when available. Files containing only time, current range, power and resistance cannot reconstruct a CV reliably. The parser now names missing channels and asks for a complete export from the original measurement.

parse_file

```
e.parse_file(filepath, options=None)
```

Read the standardized parser contract without constructing an analysis object.

Parameters

Parameter	Meaning and default
filepath	File path; required.
options	Import controls; None uses defaults.

Returns

ParseResult with data, units, technique, software, metadata, raw_metadata, warnings, source and parser.

Limitations and behavior

- Use object accessors after promotion when reference shifts or processing should be applied.

Full options: e.describe_options("get_data").

Example

```
parsed = e.parse_file("path/to/trace.mpt", {"print": False})
print(parsed.warnings)
```

Selected options

Option	Default	Meaning
allow self reference	True	Allow a CV to be considered as its own reference candidate during automatic reference shifting.
peak prominence	None	Minimum peak prominence for reference extrema on each segment; None estimates a separate threshold from each segment's search region.
reference file	None	Explicit reference file used by reference mode 'file'; no file matching is inferred.

Option	Default	Meaning
reference guess	auto	'auto' locates the reference wave automatically and raises when multiple adjacent-segment couples are indistinguishable; a numeric value ranks physical pairs near that potential.
reference keyword	None	Single keyword used by reference mode 'keyword' to choose reference files.
reference keywords	['Fc', 'ferrocene', 'decamethylferrocene', 'DmFc', 'cobaltocene', 'CoCp2']	Reference keywords tried by reference mode 'auto'; the first successful keyword defines the shift source.
reference label	Fc/Fc+	Axis label used after reference shifting.
reference map	None	Explicit target-to-reference object index mapping; overrides automatic reference assignment for those objects.
reference max delta ep	0.2	Largest allowed E_{pa} - E_{pc} separation in V for an adjacent-segment reference pair.
reference mode	auto	'auto' searches imported files with reference keywords; 'keyword', 'file', and 'manual' use the corresponding explicit reference source. Automatic peak pairing requires adjacent, opposite-direction CV segments.
reference offset	None	Manual reference potential offset.
reference smooth	True	Smooth each complete CV segment independently before reference extrema are detected; smoothing never crosses a switching vertex.

Complete registered options: e.describe_options("get_data").

EC-Lab export requirements

A CV export must contain measured potential (Ewe/V or <Ewe>/V) and current (I/mA or <I>/mA); include time/s when available. Files containing only time, current range, power and resistance cannot reconstruct a CV reliably. The parser now names missing channels and asks for a complete export from the original measurement.

filter

```
e.filter(object_list, filter_keys=None, options=None)
```

Select objects with a criteria dictionary, or inspect available values by omitting criteria or passing field names.

Parameters

Parameter	Meaning
object_list	Flat or grouped echem objects.
filter_keys	Dictionary of selection criteria; None or "available" inspects all fields; a list of field names inspects those fields.
options	FilterOptions or dictionary; defaults apply when omitted.

Selected options

Option	Default / type	Purpose and choices
logic	None str or None	Logical rule used to combine top-level filter criteria. For membership keys such as compounds, concentrations, and species, lists require all values by default; use {'any': [...]} or {'all': [...]} inside the filter key for per-key logic. Choices: AND, OR.
mode	'include' str	Mode selector: include or exclude objects matching the filter criteria. Choices: include, exclude.
sig figs	'auto' int or str or dict[str, int or str or None] or None	Numeric metadata matching precision: 'auto' (default) uses 3 significant digits for scan windows and 5 for other numeric fields; None matches exactly; integers 1-17 set significant digits. A field mapping applies precision only to named continuous fields (omitted fields match exactly). Decimal rounding buckets are shared by filter/sort/group; they do not infer instrument resolution or alter stored data. Categorical fields and integer IDs remain exact.
pretty print	True bool	Use rich table display when printing object lists or summaries. False uses plain-text output when print is True.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
print conditions	True bool	Print shared conditions, electrodes, and reference summaries above object tables. Known electrodes remain in table columns when this is False.
candidate limit	12 int or None	Maximum candidate values printed per field during inspection or after a failed filter; None prints all. Inspection returns all values regardless of this limit.
display columns	[] object	Additional metadata columns in object/conditions tables, or 'all'. Does not alter imported data or analysis result columns.

Complete registered options: e.describe_options("filter").

Returns

Dictionary criteria return selected objects, preserving supplied groups. Inspection returns a dictionary of all available values, even when the printed preview is limited. An empty dictionary selects all objects; an empty field list returns an empty dictionary. Unknown inspection fields raise an error.

Examples

```
selected = e.filter(objects, {"Solvent": "ACN", "WE": "GC"})
choices = e.filter(objects, ["solvent", "scan window", "RE"])
all_choices = e.filter(objects, "available", {"print": False})
```

Keys accept case and underscore aliases. Name matching is case-insensitive; solvent matching recognizes chemical aliases without rewriting stored labels. Subfolder criteria accept either slash direction; use raw Python strings for backslashes. Exact-folder matching is the default; subfolder recursive=True includes descendants.

A scalar scan-window criterion matches either endpoint. A two-number pair matches both endpoints regardless of order; nested pairs select alternative windows.

```
one_end = e.filter(objects, {"scan window": -1.0})
pair = e.filter(objects, {"scan window": [-1.0, 0.5]})
either = e.filter(objects, {"scan window": [[-1.0, 0.5], [-1.5, 0.5]]})
```

sort

```
e.sort(object_list, sort_keys, options=None)
```

Sort a flat list by metadata, in place.

Parameters

Parameter	Meaning and default
object_list	Mutable list of eCAT objects.
sort_keys	Key string or list, ordered by priority.
options	SortGroupOptions or dict: print, sig figs and display controls.

Returns

The same list, reordered. Equal keys preserve input order.

Limitations and behavior

- Use a shallow list copy to preserve original ordering; this does not duplicate the objects.

Full options: e.describe_options("sort_group").

Example

```
ordered = e.sort(objects.copy(), "scan rate", {"print": False})
```

Selected options

Option	Default	Meaning
sig figs	auto	Numeric metadata matching precision: 'auto' (default) uses a deterministic 5-significant-digit fallback; None matches exactly; integers 1-17 set significant digits. A field mapping applies precision only to named continuous fields (omitted fields match exactly). Decimal rounding buckets are shared by filter/sort/group; they do not infer instrument resolution or alter stored data. Categorical fields and integer IDs remain exact.
pretty print	True	Use rich table display when printing object lists or summaries. False uses plain-text output when print is True.
print	True	Whether to emit output. False suppresses output; it is independent of pretty print.
print conditions	True	Whether to include condition columns in printed object tables.

Complete registered options: e.describe_options("sort_group").

group

```
e.group(object_list, group_keys, options=None)
```

Collect objects sharing metadata into groups.

Parameters

Parameter	Meaning and default
object_list	Flat object sequence.
group_keys	Key string or list.
options	SortGroupOptions or dict; same numeric matching policy as filter.

Returns

A nested list of groups containing the original objects.

Limitations and behavior

- Grouping does not copy signal arrays or certify that all other experimental conditions match.

Full options: e.describe_options("sort_group").

Example

```
groups = e.group(objects, ["gas", "scan rate"], {"print": False})
```

Selected options

Option	Default	Meaning
sig figs	auto	Numeric metadata matching precision: 'auto' (default) uses a deterministic 5-significant-digit fallback; None matches exactly; integers 1-17 set significant digits. A field mapping applies precision only to named continuous fields (omitted fields match exactly). Decimal rounding buckets are shared by filter/sort/group; they do not infer instrument resolution or alter stored data. Categorical fields and integer IDs remain exact.
pretty print	True	Use rich table display when printing object lists or summaries. False uses plain-text output when print is True.
print	True	Whether to emit output. False suppresses output; it is independent of pretty print.
print conditions	True	Whether to include condition columns in printed object tables.

Complete registered options: e.describe_options("sort_group").

Known solvent aliases group by chemical identity by default: ACN and MeCN form one group while retaining their stored labels. Set solvent grouping="label" to separate entered labels. Plot legends and colorbar headings use the most frequent supplied spelling within a combined solvent identity; ties resolve consistently. WE, RE and CE alias working electrode, reference electrode and counter electrode; missing electrodes sort last. A physical reference electrode remains separate from the potential reporting reference.

Filter inspection and numeric matching

Use e.filter(objects) for all available metadata or e.filter(objects, ["scan rate", "scan window"]) for selected fields. The former standalone inspection helper has been removed. Returned choices preserve stored precision and labels; they are not rounded matching keys.

Automatic sig figs uses three significant figures for scan windows and five for other continuous metadata. None selects exact matching; integers 1–17 select explicit significant figures. A field mapping applies only to named fields, leaving omitted fields exact. This uses rounding buckets, not a distance tolerance or an inferred instrument resolution. Measured potentials, currents and metadata remain unchanged. Display sig figs controls formatting separately.

```
same_window = e.filter(
    objects, {"scan window": [-1.0, 0.5]},
    {"sig figs": {"scan window": 3}, "print": False},
)
```

sort_and_group

```
e.sort_and_group(object_list, sort_keys=None, group_keys=None, options=None)
```

Sorts echem objects by selected metadata keys, then groups objects sharing selected grouping keys. Chrono objects can be sorted/grouped by SI-valued keys such as applied potential, sample interval, run time, final charge, cathodic current, anodic current, high potential limit, and low potential limit.

Parameters

Parameter	Meaning
sort_keys	one key or a list such as ['gas', 'compounds', 'timestamp'], ['applied potential'], or ['final charge'].
group_keys	one metadata key or a list. If omitted, sort_keys also become the grouping keys; if sort_keys is omitted, group_keys also set the sort order.
options (dict or SortGroupOptions)	display controls.

Key options

Option	Default / type	Purpose and choices
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
pretty print	True bool	Use rich table display when printing object lists or summaries. False uses plain-text output when print is True.
print conditions	True bool	Whether to include condition columns in printed object tables.
sig figs	auto int or str or dict[str, int or str or None] or None	Numeric metadata matching precision: 'auto' (default) uses a deterministic 5-significant-digit fallback; None matches exactly; integers 1-17 set significant digits. A field mapping applies precision only to named continuous fields (omitted fields match exactly). Decimal rounding buckets are shared by filter/sort/group; they do not infer instrument resolution or alter stored data. Categorical fields and integer IDs remain exact.

Returns

list[list[echem]] when either key argument is provided. If both are omitted, the original list is returned with a message. Sorting is performed on a shallow list copy; the contained objects are shared.

Example

```
groups = e.sort_and_group(
    objects,
    sort_keys=["gas", "compounds", "timestamp"],
    group_keys=["type"],
)
```

group_summary

```
e.group_summary(object_list, group_keys=None, options=None)
```

Summarizes a flat list of echem objects or an existing grouped list, returning one pandas DataFrame row per group.

- object_list: flat echem object list or nested groups produced by group() / sort_and_group().
- group_keys: optional key or list of keys. When provided, it overrides options['group keys'] and groups internally using group(..., {'print': False}).
- options (dict or GroupSummaryOptions): 'group keys', 'columns', 'print', 'pretty print', and 'sig figs'.
- The summary includes group key values, total object count, CV/CA/CP/DPV counts when present, and distinct counts plus value lists for requested or varying metadata columns.
- Concentration grouping follows group() exactly, including grouping by the full parsed concentrations list.

Returns

pandas.DataFrame with display-ready summary columns.

```
summary = e.group_summary(
    objects, group_keys="concentrations", options={"columns": ["scan rate"]}
)
```

Selected options

Option	Default	Meaning
columns	[]	Legacy import hint retained for custom-reader compatibility; built-in text readers infer columns from the file. Not a display selector.
group keys	None	Metadata field or fields used to group objects before summarizing.
pretty print	True	Use rich table display when printing object lists or summaries. False uses plain-text output when print is True.
print	True	Whether to emit output. False suppresses output; it is independent of pretty print.
sig figs	3	Significant figures used for reported values.

Complete registered options: e.describe_options("group_summary").

save_data

```
e.save_data(object_list, options=None)
```

Exports echem object data to CSV or to an eCAT Excel workbook. CSV is the flat external table format. Excel uses a manifest sheet plus one sheet per object class for human-editable, round-trip-friendly data. Referenced CVs export both the stored Potential axis and the active Potential vs ... axis by default when a reference shift is stored.

Parameters

Parameter	Meaning
object_list	echem objects or groups.
options	output folder, filename, format, metadata columns, data columns, share x axes, and unit conversion controls. Use {'format': 'xlsx'} for Excel export. metadata columns is additive like show(); data columns is strict and can be 'all', 'stored', or an exact list of exported data columns.

Returns

pd.DataFrame for CSV export, or a dict of workbook DataFrames for Excel export.

Example

```
df = e.save_data(
    groups,
    {"folder path": r"C:/data/processed", "file name": "grouped_cv_data"},
)
workbook = e.save_data(
    objects,
    {
        "format": "xlsx",
        "folder path": r"C:/data/processed",
        "file name": "experiment_export",
        "metadata columns": ["reference source"],
        "data columns": "all",
        "share x axes": True,
    },
)
```

Selected options

Option	Default	Meaning
data columns	'all'	Data columns included in Excel data sheets: 'all', 'x', 'y', or an explicit column/list of columns. Choices: all, x, y.
metadata columns	'used'	Metadata columns included in the export manifest: 'used', 'all', or an explicit column/list of columns. Choices: used, all.
file name	'output'	Base output filename without extension; eCAT appends .csv or .xlsx from the selected format.
folder path	'.'	Folder where the exported file should be written.
format	'csv'	Output format. 'csv' writes one wide table; 'xlsx'/'excel' writes a workbook-style manifest and class-specific data sheets. Choices: csv, xlsx, excel.
x unit	None	Optional x-axis unit override applied to exported x columns when conversion is available.
y unit	None	Optional y-axis unit override applied to exported y columns when conversion is available.
print	True	Whether to emit output. False suppresses output; it is independent of pretty print.
share x axes	True	When True, Excel export combines objects with equivalent x axes into shared x-axis blocks.
sig figs	6	Significant figures used in exported metadata/manifest values.
units	'{}'	Column-specific unit overrides, as a dict mapping column names to target units.

Complete registered options: e.describe_options("save_data").

get_data_from_excel

```
e.get_data_from_excel(file_path, options=None)
```

Imports eCAT Excel workbooks when a manifest sheet is present. If no manifest exists, falls back to curated single-workbook CV header parsing using curve names in the first header row, axis labels in the second header row, and the same filename/header metadata parser settings as other eCAT imports.

Parameters

Parameter	Meaning
file_path	Excel workbook path.
options	import/object construction controls.

Key options

Option	Default / type	Purpose and choices
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
troubleshoot	False bool	Print import diagnostics plus reference segment/candidate tables and display the segment-colored reference-pair plot.
name alterations	None object or None	Filename text replacements applied during import.
electrode area	0 float	Electrode area in cm^2 assigned to imported objects for later current-density plotting, normalization, and kinetic analysis.
electrode diameter	0 float	Electrode diameter in cm used to compute electrode area when electrode area is not provided.

Use get_data_from_excel() for Excel import. Manifest workbooks can contain CV, CA, CP, and DPV objects; fallback header parsing returns CV objects.

Returns

list of eCAT objects. Manifest workbooks can include CV, CA, CP, and DPV objects; fallback header parsing returns CV objects.

Example

```
objects = e.get_data_from_excel("processed_curves.xlsx", {"print": True})
```

Objects and result ownership

Object	Use and creation
echem	Generic experiment; e.echem.from_file() promotes a supported technique automatically.
cv	Cyclic voltammetry with segment-aware peak and wave analysis.
ca	Chronoamperometry and charge/current/rate metrics.
cp	Chronopotentiometry with cycle summaries and plots.
dpv	Differential pulse voltammetry and overlap fitting.
ParseResult	Parser evidence and warnings; obtain with e.parse_file() or obj.parse_result.

Object	Use and creation
AnalysisResult	Dictionary-compatible analysis output; use .table for DataFrame operations.
CVAnalysisResult	Single-CV named values, .primary, .table and .diagnostics.
ChronoAnalysisResult	Chrono named values and .table; do not assume a .primary attribute.
ScatterFitResult	Fit tables and metadata; tuple unpacking is deliberately unsupported.

echem.metadata

```
echem.metadata()
```

Inspect a reusable metadata snapshot.

Returns

Independent dictionary of source/provenance, reference, units, processing and technique statistics.

Limitations and behavior

- obj.stats() returns technique-specific statistics. obj.data is stored data; it may not include a virtual reference-corrected axis.

Example

```
metadata = obj.metadata()
stats = obj.stats()
e.show(obj)
```

echem.xy

```
echem.xy(options=None)
```

Read the active coordinate axes together.

Parameters

Parameter	Meaning and default
options	Axis selection and view controls. x() and y() accept the same dictionary.

Returns

(x, y): pandas Series on the base echem implementation; cv.xy() returns NumPy arrays. x() and y() normally return Series; one column=False can expose multi-column data. These accessors do not uniformly apply CV segment selection; use analysis_segment_data() for a selected sweep.

Limitations and behavior

- Reference correction, normalized columns and unit metadata affect the active axes. Do not assume obj.data contains the same coordinates as obj.x().

Full options: e.describe_options("plot").

Example

```
potential, current = cv_obj.analysis_segment_data({"segment": 3})
raw_potential = cv_obj.x({"x axis": "Potential"})
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("echem.xy").

Object settings and unit conversions

Method	Arguments and return	Effect or limitation
obj.stats()	No arguments; returns dict.	Technique-specific values; units are in object metadata or documented key labels.
obj.set_electrode_area(area)	area in cm^2; returns None.	Mutates area metadata. Prefer setting the known area during import.
obj.set_temperature(temp)	Temperature in K; returns None.	Mutates temperature metadata; does not re-run earlier analyses.
obj.current_to(unit)	Unit such as "uA"; returns self.	Converts stored current columns in place and updates units. Prefer plot y unit for display only.
obj.to_density()	No arguments; returns self.	Adds Current Density in place using nonzero electrode area in cm^2.
obj.invert_current()	None.	Changes stored current polarity. Prefer a plot-view convention when only presentation should change.

Method	Arguments and return	Effect or limitation
obj.has_reference_shift()	No arguments; returns bool.	Whether a virtual reference shift can be applied to stored Potential.
obj.reference_axis_name()	No arguments; returns str.	Name of the active reference-potential axis.
obj.to_min()	No arguments; returns None.	Legacy helper handles a first column containing Time/sec. Prefer x unit="min" for standard Time columns.

Parser construction, label-formatting and axis-scaling internals are implementation helpers, not additional stable notebook workflows. Use import/accessor/plot APIs instead of calling undocumented parser internals directly.

show

```
e.show(value, options=None)
```

Display an object, collection or showable analysis result.

Parameters

Parameter	Meaning and default
value	One object, a flat list, nested groups, or a result with show().
options	print/display settings; return=True requests the displayed table where supported.

Returns

None by default; a table when return=True is supported by the dispatched display helper.

Limitations and behavior

- show_objects(object_list, options=None) handles a flat list; show_groups(groups, options=None) handles already grouped lists. Neither performs analysis or grouping.

Example

```
e.show(objects)
table = e.show_objects(objects, {"return": True})
e.show_groups(groups)
```

Selected options

Option	Default	Meaning
return	False	Request a table where the dispatched helper supports it.

For echem objects and collections, show() displays whenever called; print=False does not suppress these paths. Options are dispatched to the selected helper, so result-specific print behavior is separate. Use metadata() for a silent object snapshot.

show_objects

```
e.show_objects(object_list, options=None)
```

Display a flat collection of objects; use e.show() for grouped collections. options controls display columns, units and significant figures. Returns None by default or a DataFrame with return=True (an empty collection returns an empty list). Stored metadata and arrays are unchanged. show_objects displays when called; print=False is not a suppression option here. Use filter(..., {"print": False}) to inspect quietly.

Option	Default	Meaning
return	False	Return the displayed DataFrame in addition to displaying it.
pretty print	True	Rich display when supported; False prints plain text.
print conditions	True	Show common conditions, electrode and reference summaries.
columns	[]	Additional metadata columns; distinguishing columns remain visible.
sig figs	3	Significant figures for display, independent of filter matching.

```
e.show_objects(objects, {"columns": ["name", "scan rate", "WE", "RE"]})
```

Shared WE/RE/CE values appear once in an [Electrodes] line when print conditions=True. Differing electrodes stay in per-file table columns. Use columns to request display columns; import columns controls input file parsing and is a separate option. A folder path is not prepended to an already relative subfolder.

AnalysisResult.to_dataframe

```
AnalysisResult.to_dataframe(kind='table')
```

Access an analysis table without confusing dictionary keys with columns.

Parameters

Parameter	Meaning and default
kind	"table" (default), "fit table", "raw" or "transformed"; data/primary are table aliases.

Returns

The selected DataFrame, or None if that optional table is absent. The returned table is not copied.

Limitations and behavior

- to_csv(path_or_buf=None, *args, **kwargs) follows pandas: returns CSV text without a path and usually None with a path.
- to_excel(excel_writer, *, sheet_name="table", index=False, include_metadata=True, **kwargs) returns None and writes the primary table plus available metadata sheets. Both exports require a primary table.
- result.show({"return": True}) returns the displayed table. .figure/.axes/.figures are populated only by workflows that collect figures.

Example

```
frame = result.to_dataframe()
result.to_csv("analysis.csv", index=False)
result.to_excel("analysis.xlsx")
```

Plotting

echem.plot

```
echem.plot(options=None, **mpl_kwargs)
```

Plots one echem object using its selected x/y data. CVs can plot selected segments; CA current plots include 0 on the y axis by default, and CA/CP plots use auto titles consistent with CV/DPV. CA objects can add cumulative charge on a secondary axis with {'plot charge': True}.

- Use {'charge color': 'tab:green'} or another Matplotlib color to color the secondary charge line, right-axis label, ticks, and spine together. ca.charge() plots cumulative charge in black unless a color is supplied, accepts regular plot options such as title, and can plot baseline-corrected charge when baseline correction options are supplied.
- CP cycling_plot() overlays capacity/efficiency performance metrics and accepts {'cycles': ...} to select points by cycle number. CP plot_cycles() shows selected cycle traces and supports gradient cycle coloring with the same custom colorbar style used by multiplot.

Parameters

Parameter	Meaning
options (dict or PlotOptions)	axis, unit, segment, minor-tick, and styling controls.

- Use 'derivative': 1 or 2 to plot Savitzky-Golay derivative views; derivative is a plotting view and does not mutate the stored current data. Current-vs-potential derivative plots label the y axis as d_i/dE or d^2i/dE^2 with composed units such as microampere per volt.
- For current-density views, set 'y axis': 'current density' with an electrode area. eCAT reports the plot-time axis as A/cm^2 by default and auto-scales the numerator for readable values, such as microampere per cm^2. The 'y unit' option accepts current-only units such as 'uA', area-only shorthand such as 'mm', 'mm^2', or '/mm^2', and full units such as 'uA/mm^2'. Full current-density units are used exactly, while area-only shorthand keeps numerator auto-scaling.
- CV segment coloring: 'segment color mode' defaults to 'auto', which uses grouped discrete-gradient colors when a CV has more than 3 plotted segments; set it to 'off', 'discrete', 'discrete gradient', or 'continuous gradient' to choose explicitly.

cv.plot() creates a new figure by default. Use new plot=False to overlay intentionally. e.plotting_style("publication") selects larger export-oriented styling; e.plotting_style("notebook") restores the notebook profile and e.plotting_style(False) restores Matplotlib defaults.

For CV potential programs, cv.plot_program(options=None) plots potential versus time. If the CV has a time column, eCAT uses it; otherwise it reconstructs elapsed time from scan rate. Quiet time is included by default as a negative-time hold at the starting potential; pass {'plot quiet time': False} to omit it.

Use cv.trim({'potential window': [start, stop]}) to return a copied CV object trimmed to a displayed potential window before plotting or downstream analysis. cv.trim accepts the usual eCAT options-dict style or TrimOptions; convenience forms such as cv.trim([start, stop], window_mode='allow') are also supported.

- The default window mode is 'expand', which preserves a connected CV waveform when a pointwise window would disconnect the scan history; use 'allow' for pointwise trimming or 'strict' to reject disconnected windows. The original CV is unchanged unless inplace=True is supplied.
- Use 'minor ticks' to control per-axis minor ticks: True uses the eCAT default, False disables minor ticks, and an integer sets the AutoMinorLocator subdivision count.
- Use 'segment color groups' to group segment colors; an integer is a minimum group size, while explicit groups like [[1, 2], [3, 4, 5]] preserve original segment numbers in labels.
- Segment colorbars use the same custom inset legend style and adaptive placement as multiplot, with 'Segments' as the context line and numeric segment labels on the bar; 'legend': 'auto' only displays a legend/colorbar when there is more than one entry, and 'legend loc': 'auto' chooses a low-overlap location.
- Set 'colorbar style' to 'auto', 'discrete', or 'continuous'; 'auto' follows the segment color mode, so 'discrete gradient' uses a continuous colorbar while 'discrete' forces swatches.
- **mpl_kwargs: passed through to matplotlib plotting where supported.

Key options

Option	Default / type	Purpose and choices
x axis	None str or None	Column or axis used as x data.

Option	Default / type	Purpose and choices
y axis	None str or None	Column or axis used as y data.
x unit	'auto' str or None	'auto' auto scales from the displayed data and object units; explicit units force the requested x-axis unit.
y unit	'auto' str or list[str or None] or tuple[str or None, str or None] or None	Current display unit, or [current unit, charge unit] for a charge overlay. A scalar controls current only and leaves charge on auto scaling.
plot convention	'IUPAC' str	Electrochemical plotting convention for axis orientation and signs. Choices: US, IUPAC.
plot segment	None int or None	Segment to emphasize or plot.
plot segments	None list[int] or int or None	Segments to emphasize or plot.
color	'black' str or None	Primary plot color.
label	None str or None	Label for a plotted trace or analysis output.
legend	'auto' bool or str	'auto' suppresses single-entry legends and shows legends when multiple plotted entries need labels.
title	True bool or str	Plot title text or title mode. 'auto' derives a concise title from object metadata.
new plot	False bool	Create a new figure before plotting.
ip0	None float or list[float] or None	Non-catalytic reference peak current.

Returns

matplotlib.axes.Axes for single-panel plots. Access the figure as ax.figure.

Example

```
cv.plot(
    {
        "plot segments": [1, 2],
        "y axis": "i/ip0",
        "ip0": 2e-6,
        "color": "tab:blue",
    }
)
cv.plot(
    {"segment color mode": "discrete gradient", "segment color groups": 2}
)
```

multiplot

```
e.multiplot(echem_list, options=None)
```

Plots many echem objects on one figure with shared unit scaling, automatic labels, optional gradients, and adaptive legend/colorbar behavior.

Parameters

Parameter	Meaning
echem_list	echem objects to overlay.

- Set 'legend mode': 'discrete' with 'legend outside': True to place a normal trace legend outside the axes.
- Use 'min gradient entries' to choose how many distinct gradient values are required before auto legend mode uses gradient colors and a colorbar; duplicate traces at the same value do not increase the count, and smaller groups remain discrete. This option also routes through FOWA plot-all multiplots.

Use 'deduplicate labels' to distinguish repeated multiplot labels after label alterations. The default False leaves labels unchanged but warns when duplicates are detected and suggests {'deduplicate labels': True}. Set it to True to append only differing scan window and segment metadata, omitting fields that match; if labels still collide, eCAT appends replicate numbering. Pass a valid metadata key or list of keys to choose the metadata used; invalid keys print the valid options.

- Mole-fraction colorbars use χ(species) endpoint labels and do not add the leading '+' used for concentration additions.
- Gradient-colored multiplots separate color mapping from colorbar display: 'gradient ...' options choose how trace metadata maps to colors, while 'colorbar ...' options control the displayed colorbar legend. Use 'colorbar tick labels' ('endpoints', 'all', or 'none') and 'colorbar trace ticks' to control tick text and per-trace tick marks.
- With 'gradient scale': 'auto', scan-rate gradients and positive concentration gradients use log spacing; percent and equivalent concentration series follow that same concentration-gradient default, while zero-concentration entries are treated as background traces rather than zero-valued colorbar endpoints. 'gradient reverse' reverses trace color assignment; 'colorbar reverse' flips only the displayed colorbar direction.
- options (dict or MultiplotOptions): shared axis, label, color, legend, and layout controls.

Key options

Option	Default / type	Purpose and choices
x axis	None str or None	Column or axis used as x data.
y axis	None str or None	Column or axis used as y data.
x unit	'auto' str or None	Requested x-axis unit.
y unit	'auto' str or list[str or None] or tuple[str or None, str or None] or None	Requested y-axis unit.
labels	None list[str] or None	Explicit labels for plotted objects. If omitted, labels are generated from object metadata.
label alterations	None object or None	Text replacements applied to generated labels.
title	'auto' bool or str	'auto' builds a shared title from metadata common to the plotted objects.

Option	Default / type	Purpose and choices
subtitle	'auto' bool or str or None	'auto' builds shared context from metadata common to the plotted objects.
color mode	'auto' str	'auto' detects scan-rate or concentration gradients and colors those groups by gradient; remaining traces use discrete colors. Choices: auto, gradient, discrete.
legend mode	'auto' str	'auto' uses colorbar legends for detected gradients and discrete labels for non-gradient traces. Choices: auto, colorbar, discrete.
legend loc	'auto' str	'auto' uses Matplotlib's best location for in-axes legends.
legend outside	False bool	Place the legend outside the axes.
colorbar height scale	1.0 float	Scale factor for colorbar height in gradient legends.
gradient by	'auto' str	'auto' first looks for scan-rate gradients, then concentration gradients. Choices: auto, scan rate, concentration.
gradient species	'auto' str	'auto' uses the species whose concentration varies within the detected concentration-gradient group.
gradient scale	'auto' str	'auto' uses log scale for scan rate and concentration gradients, and linear scale otherwise. Choices: auto, linear, sqrt, log, index.
offset	0 float or list[float]	In displayed y-axis units, a numeric value applies a constant vertical step multiplied by trace index; a list supplies one explicit absolute vertical offset per trace.

Returns

matplotlib.axes.Axes for the shared multiplot panel.

Example

```
e.multiplot(
    cvs,
    {
        "y axis": "i/ip0",
        "ip0": [1e-6, 1.2e-6],
        "legend mode": "discrete",
        "legend outside": True,
    },
)
```

multimultiplot

```
e.multimultiplot(echem_groups, options=None)
```

Creates one multiplot per group of echem objects.

Parameters

Parameter	Meaning
echem_groups	list of groups or a flat list.

Parameter	Meaning
options (dict or MultiMultiplotOptions)	group titles/subtitles plus multiplot controls.

Key options

Option	Default / type	Purpose and choices
titles	'auto' list[str] or str or None	'auto' derives one title per panel/group from grouped-object metadata.
subtitles	'auto' list[str] or str or None	'auto' derives one subtitle per panel/group from grouped-object metadata.
legend	True bool	Whether to show a plot legend.
legend mode	'auto' str	Legend mode for discrete or gradient color encodings. Choices: auto, colorbar, discrete.
labels	None list[str] or None	Explicit labels for plotted objects.
color mode	'auto' str	Color assignment mode for multi-trace plots. Choices: auto, gradient, discrete.
gradient by	'auto' str	Metadata field used to assign gradient colors. Choices: auto, scan rate, concentration.
y axis	None str or None	Column or axis used as y data.
y unit	'auto' str or list[str or None] or tuple[str or None, str or None] or None	Requested y-axis unit.

Returns

None in the current implementation. Figures are created as side effects. To retain each Axes explicitly, call e.multiplot(group, options) for each group.

Example

```
e.multimultiplot(groups, {"titles": "auto", "legend outside": True})
```

multi_scatterplot

```
e.multi_scatterplot(datasets, options=None)
```

Overlays labeled rate, peak-current, or peak-potential result tables. AnalysisResult or ScatterFitResult inputs reuse existing fit metadata for fit-line overlays.

Parameters

Parameter	Meaning
datasets (dict)	labels mapped to DataFrames or AnalysisResult objects.
options (dict or MultiScatterplotOptions)	column, style, legend, color, title, and fit-line display controls.

- Column selection: 'x column', 'y column', and 'y columns' select from existing result-table columns; they do not recompute upstream fits. Stored fit overlays are reused only when compatible with the selected columns. For example, plotting raw k_{obs} from a fit_rate result that used a non-raw y mode requires a matching upstream fit_rate(..., {'y mode': 'raw'}) result or 'plot fit': False.

Key options

Option	Default / type	Purpose and choices
x column	'auto' str or int or None	'auto' auto prefers transformed/raw x columns and falls back to the first sensible x column. Explicit columns control plotted points.
y column	'auto' str or int or None	'auto' auto prefers transformed, metric, kobs, TOFmax, ip, then Ep result columns. Explicit columns control plotted points.
y columns	None object or None	Explicit y columns to plot; when omitted, y-column auto-resolution is used.
metric	None str or None	Preferred metric column used during y-column auto-resolution.
plot style	'scatter' str	Plot style such as scatter or line. Choices: scatter, line, line+markers.
plot fit	True bool	Whether to overlay fitted curves or lines.
legend	True bool	Whether to show a plot legend.
labels	None list[str] or None	Explicit labels for plotted objects.
label alterations	None object or None	Text replacements applied to generated labels.
title	'auto' bool or str	Plot title text or title mode.
subtitle	'auto' bool or str or None	Plot subtitle text or subtitle mode.
fit linestyle	'--' str	Line style for plotted fit lines.
fit linewidth	1 int or float	Line width for plotted fit lines.
fit alpha	1 int or float	Alpha transparency for plotted fit lines.

Returns

ScatterFitResult, an AnalysisResult child, with figure, axes, table, fit_table, and summary metadata.

- By default, multi_scatterplot displays the reused fit_table, including slopes and intercepts, when AnalysisResult inputs provide reusable fits. Set 'print': False to suppress this display.

Example

```
summary = e.multi_scatterplot(
    {"Fe only": rate_result, "cyclen": other_result}, {"legend": True}
)
```

- Use options['x column'], options['y column'], and options['y columns'] to choose exactly which existing result columns are plotted. multi_scatterplot does not reinterpret upstream analysis modes; create raw, transformed, or adjusted results upstream, then select the desired columns here.
- Transformed y-axis labels preserve stored y mode information, so fractional or adjusted results display labels such as log10(k_{obs}/kobs0 - 1) instead of only log10(k_{obs}).
- When raw or adjusted data are plotted from transformed fit results, built-in transforms such as log10, ln, reciprocal, and powers are back-transformed for fit overlays when possible.

animate

```
e.animate(obj_or_list, options=None)
```

Animates one electrochemical object or a list of objects using the same plot and multiplot styling that eCAT uses for fully rendered static figures.

Parameters

Parameter	Meaning
obj_or_list	one echem/cv/ca/cp/dpv object or a list of objects.
options (dict or PlotOptions)	regular plot styling plus animation controls such as trace mode, schedule, timing mode, normalized duration, speedup, fps, stride, stagger time, end hold, cycle, include quiet time, print, pretty print, and progress. Use stride to render every nth plotted point for lighter animation previews or exports without changing the source object data.

Behavior notes

- Default animation behavior is object-aware: a single object uses trace mode = 'draw'; multiple objects use simultaneous draw playback when scan rates differ and staggered instant playback when scan rates match.
- Timing mode = 'auto' uses physical experiment timing when enough metadata are available; otherwise it falls back to normalized timing. In normalized mode, 'normalized duration' is the per-trace display time. In physical mode, 'speedup' compresses experiment time by the requested factor. Quiet time is excluded unless 'include quiet time' is True.
- Animation uses the same eCAT plot helpers as static plot()/multiplot() calls, so the fully rendered frame matches the corresponding static figure style, units, labels, and legend/colorbar behavior.
- Use result.save('movie.gif') or result.save('movie.mp4') to export the animation. GIF uses Pillow; MP4 uses Matplotlib's ffmpeg writer when available in the environment. Rendering progress is shown by default during display and export; set 'progress': False to suppress it.

Returns

AnimationResult
with .animation, .figure, .axes, .summary, .display(options=None), .show(options=None), .to_html(options=None), and .save(path, format=None, options=None).

Example

```
result = e.animate(
    cvs,
    {
        "gradient by": "scan rate",
        "legend mode": "colorbar",
        "timing mode": "normalized",
        "normalized duration": 2.0,
        "fps": 20,
        "stride": 2,
        "progress": True,
    },
)
result.save("scan_series.gif")
```

Selected options

Option	Default	Meaning
plot segments	None	Segments to emphasize or plot.
y unit	'auto'	Current display unit, or [current unit, charge unit] for a charge overlay. A scalar controls current only and leaves charge on auto scaling.
x axis	None	Column or axis used as x data.
x unit	'auto'	'auto' auto scales from the displayed data and object units; explicit units force the requested x-axis unit.
y axis	None	Column or axis used as y data.
label alterations	None	Text replacements applied to generated labels.
subtitle	'auto'	Plot subtitle text or subtitle mode. 'auto' uses object metadata that is useful context but not already in the title.
title	'auto'	Plot title text or title mode. 'auto' derives a concise title from object metadata.
color mode	'auto'	Color assignment mode for multi-trace plots. Choices: auto, gradient, discrete.
gradient by	'auto'	Metadata field used to assign gradient colors. Choices: auto, scan rate, concentration.
gradient colormap	None	Colormap used for a gradient legend.
gradient colormaps	[]	List of colormaps used for multiple gradient legends.

Complete registered options: e.describe_options("animate").

plotting_style

```
e.plotting_style(style=True)
```

Change package-level Matplotlib styling.

Parameters

Parameter	Meaning and default
style	True or "notebook" for the notebook profile; "publication" or "saveant" for named profiles; False restores Matplotlib defaults.

Returns

str: the active profile name, or "matplotlib" when disabled. This changes Matplotlib settings for subsequent plots in the Python process.

Limitations and behavior

- This is a global plotting side effect, not an object-local option.

Example

```
e.plotting_style("publication")
cv_obj.plot()
e.plotting_style("notebook")
```

echem.spectrum

```
echem.spectrum(options=None)
```

Compute Welch PSD or a calibrated one-sided FFT from a physically timed signal.

Parameters

Parameter	Meaning and default
options	SpectrumOptions or dictionary: method, column, time range, segments, window, resampling and frequency/plot controls.

Returns

AnalysisResult with frequency table, units, sampling summary, resolved-trace diagnostics, warnings and optional figures.

Limitations and behavior

- CA/CP/generic data require recorded numeric time. CV may reconstruct time from potential and scan rate within each segment; DPV may use pulse_period.
- Sampling must meet sampling tolerance (default 1%). resample=True explicitly interpolates modest irregularity. A spectrum includes waveform shape, transients and drift; it is not automatically a noise measurement.
- Frequency is Hz; PSD is signal-unit squared per Hz. FFT amplitude retains the signal unit and phase is in radians. Upper frequency is limited by Nyquist.

Full options: e.describe_options("echem.spectrum").

Example

```
spectrum = obj.spectrum({"method": "welch", "plot": False})
spectrum.table
```

Selected options

Option	Default	Meaning
detrend	'constant'	Detrending applied before Welch or FFT spectral estimation: None, constant, or linear. Choices: None, constant, linear.
include dc	False	Whether to retain the zero-frequency component in the spectrum.

Option	Default	Meaning
method	'welch'	Spectral estimation method: Welch averaging or a direct FFT. Choices: welch, fft.
nfft	None	Optional positive FFT length used for spectral estimation, limited to 2,000,000 transform points. Reduce nfft or narrow the selected trace if the limit is exceeded.
overlap	0.5	Fractional overlap between adjacent Welch windows; must be at least 0 and less than 1.
points per window	'auto'	Welch window length in points, or 'auto'. The SciPy name nperseg is accepted as an alias.
resample	False	Whether to resample nonuniform time data before spectral estimation.
sampling tolerance	0.01	Nonnegative relative tolerance used to identify nonuniform sampling intervals.
window	'hann'	Any SciPy-compatible window specification applied during Welch or FFT spectral estimation.
frequency range	None	Optional finite, increasing, nonnegative two-value frequency range retained in the result.
segments	'all'	Segment selection used before spectral estimation. 'all' includes every available segment; the singular 'segment' spelling is accepted as an alias.
time range	None	Optional finite, increasing two-value time range applied before spectral estimation. Negative time values are allowed.

Complete registered options: e.describe_options("echem.spectrum").

Equations and parameter definitions

$$X_k = \sum_{n=0}^{N-1} w_n x_n \exp\left(-i 2\pi \ \ k n / M\right)$$

FFT amplitude: $A_k = \frac{c_k |X_k|}{N G_w}; \ G_w = \frac{\sum_n w_n}{N}$

Welch density: $S_{xx}(f_k) = \frac{c_k}{K} \sum_j \frac{|X_{j,k}|^2}{f_s \sum_n w_n^2}$

Symbol	Meaning and units	Input or controlling option
x_n, w_n	Detrended signal and window weights	column, detrend, window; segments / time range select data.
N, M, K	Samples/window, FFT length, Welch window count	points per window; nfft; overlap controls Welch windows.
f_s, fk	Sampling rate and frequency (Hz)	Derived from Time; resample / sampling tolerance govern irregular sampling.

Symbol	Meaning and units	Input or controlling option
c_k	One-sided-bin multiplier	1 at DC and Nyquist when present; 2 at other positive-frequency bins.
A_k, S_{xx}	Signal amplitude; signal^2/Hz spectral density	method = fft or welch; frequency range / include dc select returned bins.

Scope and limitations

Zero padding changes the frequency grid, not true frequency resolution. Welch density and FFT amplitude have different units and normalization. Nonstationary CV signals require care; a spectral peak alone does not identify a chemical timescale.

Individual CV Data Analysis

Peak and wave call conventions

Peak defaults are exploratory. Select the segment and intended peak explicitly for final analysis. Single-feature methods can search for the largest detected feature. Paired-wave methods need opposing segments and two resolved peaks. A peak guess does not establish that two peaks belong to the same redox couple.

Complex CV analyses accept per-CV potential lists through plural aliases. describe_options shows these as one row with (s), such as 'guess potential(s)', 'tangent potential(s)', 'non-catalytic cv(s)', and 'scan rate(s)'. Use 'guess potentials', 'exact potentials', 'tangent potentials', 'peak potentials', 'non-catalytic guess potentials', or FOWA 'redox potentials' when each CV needs its own value.

- A scalar 'guess potential' keeps the existing running-guess behavior in multi-segment analyses. For paired-peak analyses such as trumpet_analysis and nicholson_analysis, a flat two-value 'guess potential' is treated as a shared paired guess when there are more than two CVs; with exactly two CVs, use nested pairs like [[g1a, g1b], [g2a, g2b]] when each CV needs a paired guess.
- If both singular and plural spellings are supplied for the same option, eCAT raises an option error.

Peak methods accept guess_or_options=None as their first argument: pass a potential guess or an options dictionary. options=None supplies an additional dictionary and keyword arguments provide supported option spellings. Select 1-based segments explicitly. The signature lists arguments; the option table lists keys inside options.

cv.peak_potential

```
cv.peak_potential(guess_or_options=None, options=None, **kwargs)
```

Finds a peak potential on the selected CV segment using an exact potential, guess potential, or automatic extrema detection.

Parameters

Parameter	Meaning
options (dict or PeakPotentialOptions)	segment, smoothing, peak-selection, plotting, and printing controls.

Key options

Option	Default / type	Purpose and choices
segment	None int or None	CV segment to analyze.

Option	Default / type	Purpose and choices
segments	None list[int] or None	One or more CV segments to analyze.
guess potential	None float or list[float] or list[list[float]] or None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
exact potential	None float or list[float] or None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
peak prominence	None float or None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
noise window	'auto' int or str or None	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.
noise polyorder	'auto' int or str	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
plot	True bool	Whether to plot the result.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
plot all	False bool	Whether to show diagnostic or child plots.
troubleshoot	False bool	Print additional troubleshooting output.

When overlaying peak-current diagnostics on an existing cv.plot() or multiplot() axes, set {'plot': True, 'plot CV': False, 'new plot': False}. This draws the peak marker, tangent baseline, and current-distance line without redrawing the underlying CV trace.

Returns

CVAnalysisResult, an AnalysisResult child that behaves like the previous dict with keys including 'Ep', 'index', and 'current'. Use result.primary for the base-unit E_p scalar, result.table for a tidy Metric/Value display table, and result.show() for pretty or plain printing.

Peak-potential diagnostics accept plot-view options such as 'derivative', so cv.peak_potential({'derivative': 1, ...}) can overlay the selected peak on a derivative trace while keeping the peak-picking analysis on the selected CV data.

Peak selection defaults to 'peak kind': 'both' because cathodic/anodic current sign conventions vary. Set 'peak kind': 'infer' to choose maxima when the selected current rises toward the feature and minima when it falls, or set 'max'/'min' explicitly. With a guess potential, automatic peak prominence is estimated from a local window around the guess when possible and falls back to the global noise estimate; an explicit 'peak prominence' overrides this. Set 'noise window': None to disable peak-finding smoothing.

Example

```
peak = cv.peak_potential({"segment": 1, "guess potential": -2.05})
Ep = peak["Ep"]
idx = peak["index"]
peak.table
```

Equations and parameter definitions

E_p = E at the selected local peak or requested exact potential.

Symbol	Meaning and units	Input or controlling option
E, i	Potential (V), current (A unless another signal axis is selected)	Selected data; segment / segments, x axis and y axis.
Ep	Peak potential (V); extraction only, not tangent-corrected peak current.	guess potential / exact potential; peak prominence; noise window / noise polyorder.

Scope and limitations

Peak location is an extraction algorithm, not a kinetic fit. Direction, smoothing, prominence and search choices can change which peak is selected. DPV has no CV sweep-pair interpretation.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

cv.peak_current

```
cv.peak_current(guess_or_options=None, options=None, **kwargs)
```

Calculates tangent-corrected peak current for the selected CV peak.

Parameters

Parameter	Meaning
options (dict or PeakCurrentOptions)	peak selection plus tangent baseline controls.

- Peak-current fallback: if peak_potential() cannot locate a local extremum, peak_current() defaults to 'peak fallback': 'highest current', which uses the largest absolute current in the selected segment and records 'peak source' as 'highest current fallback'. Use 'peak fallback': 'guess potential' to treat 'guess potential' as an exact potential, or 'peak fallback': None to keep strict peak-finding errors.

Key options

Option	Default / type	Purpose and choices
segment	None int or None	CV segment to analyze.
segments	None list[int] or None	One or more CV segments to analyze.
guess potential	None float or list[float] or list[list[float]] or None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
exact potential	None float or list[float] or None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.

Option	Default / type	Purpose and choices
tangent potential	None float or list[float] or None	Manual tangent anchor potential; when omitted, eCAT anchors from the resolved tangent region. In complex CV analyses, the plural alias 'tangent potentials' accepts per-CV values.
tangent range	'auto' str or float or list[float] or tuple[float, float]	'auto' chooses a pre-peak baseline region from derivative/curvature behavior before the peak.
tangent min points	None int or None	Minimum points for tangent fitting; when omitted, eCAT derives a minimum from the pre-peak data length.
percent threshold	None float or None	Percent threshold used in peak or tangent selection.
plot	True bool	Whether to plot the result.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
troubleshoot	False bool	Print additional troubleshooting output.

When overlaying peak-current diagnostics on an existing cv.plot() or multiplot() axes, set {'plot': True, 'plot CV': False, 'new plot': False}. This draws the peak marker, tangent baseline, and current-distance line without redrawing the underlying CV trace.

Returns

CVAnalysisResult, an AnalysisResult child that behaves like the previous dict with keys including 'ip', 'Ep', tangent-line diagnostics, and fit indices. Use result.primary for the base-unit i_p scalar; result.table scales values to readable display units such as microamps.

Example

```
peak = cv.peak_current(
    {"segment": 1, "guess potential": -1.8, "tangent range": "auto"}
)
ip = peak["ip"]
tanline = peak["tangent line"]
peak.show()
```

Equations and parameter definitions

$$i_{\text{baseline}}(E) = a\,E + b$$

$$i_p = i(E_p) - i_{\text{baseline}}(E_p)$$

Symbol	Meaning and units	Input or controlling option
E, i	Potential (V), current (A unless another signal axis is selected)	Selected data; segment / segments, x axis and y axis.
E_p, i_p	Peak potential (V), tangent-corrected peak current (A)	guess potential / exact potential; peak prominence; noise window / noise polyorder.
a, b	Baseline slope (A/V) and intercept (A)	tangent range / tangent potential / tangent min points.

Scope and limitations

The baseline-corrected peak current differs from the raw current at E_p. peak_info also returns the width and half-peak quantities defined below.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

cv.peak_width

```
cv.peak_width(guess_or_options=None, options=None, **kwargs)
```

Measures tangent-corrected full peak width for the selected CV peak.

Parameters

Parameter	Meaning
options (dict or PeakWidthOptions)	peak selection, tangent baseline, fractional level, plotting, and printing controls.

Key options

The default 'level' is 0.5, so peak_width() reports full width at half peak current. The level must be greater than 0 and less than 1. Crossings are linearly interpolated internally and reported in scan order as 'E leading' and 'E trailing'; there is no public side, mode, or interpolate option.

Returns

CVAnalysisResult with keys including 'width', 'E leading', 'E trailing', 'Ep', 'ip', 'level', 'level current', tangent-line diagnostics, and fit indices. result.primary is the base-unit width in volts, and result.table shows the unit-scaled Metric/Value display.

Example

```
width = cv.peak_width(
    {
        "segment": 1,
        "guess potential": -1.8,
        "level": 0.5,
        "tangent range": "auto",
    }
)
full_width = width["width"]
E_leading = width["E leading"]
E_trailing = width["E trailing"]
width.show()
```

Selected options

Option	Default	Meaning
level	0.5	Fraction of the tangent-corrected peak current used for the width crossing. The default 0.5 gives full width at half peak current.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.

Option	Default	Meaning
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.
peak prominence	None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
percent threshold	None	Percent threshold used in peak or tangent selection.
tangent min points	None	Minimum points for tangent fitting; when omitted, eCAT derives a minimum from the pre-peak data length.
tangent potential	None	Manual tangent anchor potential; when omitted, eCAT anchors from the resolved tangent region. In complex CV analyses, the plural alias 'tangent potentials' accepts per-CV values.
tangent range	'auto'	'auto' chooses a pre-peak baseline region from derivative/curvature behavior before the peak.
exact potential	None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
guess potential	None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.

Complete registered options: e.describe_options("cv.peak_width").

Equations and parameter definitions

$$i_{\text{corrected}}(E) = i(E) - (aE + b)$$

$$i_{\text{corrected}}(E_{\text{cross}}) = f\, i_p$$

$$W = |E_{\text{trailing}} - E_{\text{leading}}|; \quad \text{half-peak shift} = E_p - E_{p/2}$$

Symbol	Meaning and units	Input or controlling option
E, i	Potential (V), current (A unless another signal axis is selected)	Selected data; segment / segments, x axis and y axis.
E_p, i_p	Peak potential (V), tangent-corrected peak current (A)	guess potential / exact potential; peak prominence; noise window / noise polyorder.
a, b	Baseline slope (A/V) and intercept (A)	tangent range / tangent potential / tangent min points.
f	Fraction of peak height (dimensionless)	level in peak_width; fixed 0.5 in half_peak_potential.
E_{leading}, E_{trailing}	Crossing potentials (V)	Crossings on either side of the selected peak; Ep/2 uses the leading side.

Scope and limitations

peak_width interpolates the tangent-corrected crossings. half_peak_potential instead chooses the pre-peak sample nearest half the corrected peak current. Full width requires both crossings; unresolved trailing branches can make the width unavailable while the peak remains valid.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

cv.half_peak_potential

```
cv.half_peak_potential(guess_or_options=None, options=None, **kwargs)
```

Find the leading-side potential where tangent-corrected current reaches half the selected peak current. This is distinct from the midpoint of two peak potentials returned by half_wave_potential().

Parameters

Parameter	Meaning
options	CV selection and diagnostic display controls.

Key options

Option	Default / type	Purpose and choices
segment	None int or None	CV segment to analyze.
guess potential	None float or list[float] or list[list[float]] or None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
exact potential	None float or list[float] or None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
peak prominence	None float or None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
tangent range	'auto' str or float or list[float] or tuple[float, float]	'auto' chooses a pre-peak baseline region from derivative/curvature behavior before the peak.
plot	True bool	Whether to plot the result.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
troubleshoot	False bool	Print additional troubleshooting output.

Returns

CVAnalysisResult with $E_{p/2}$, $\Delta(E_p - E_{p/2})$, peak current and tangent diagnostics, plus a display table. Potential values are in V.

Example

```
half_peak = cv_obj.half_peak_potential(
    {"segment": 1, "guess potential": -2.0, "plot": False}
)
half_peak.table
```

Equations and parameter definitions

$$i_{\text{corrected}}(E) = i(E) - (aE + b)$$

$$i_{\text{corrected}}(E_{\text{cross}}) = f\, i_p$$

$$W = |E_{\text{trailing}} - E_{\text{leading}}|; \ \text{half-peak shift} = E_p - E_{p/2}$$

Symbol	Meaning and units	Input or controlling option
E, i	Potential (V), current (A unless another signal axis is selected)	Selected data; segment / segments, x axis and y axis.
E_p, i_p	Peak potential (V), tangent-corrected peak current (A)	guess potential / exact potential; peak prominence; noise window / noise polyorder.
a, b	Baseline slope (A/V) and intercept (A)	tangent range / tangent potential / tangent min points.
f	Fraction of peak height (dimensionless)	level in peak_width; fixed 0.5 in half_peak_potential.
E_{leading}, E_{trailing}	Crossing potentials (V)	Crossings on either side of the selected peak; Ep/2 uses the leading side.

Scope and limitations

peak_width interpolates the tangent-corrected crossings. half_peak_potential instead chooses the pre-peak sample nearest half the corrected peak current. Full width requires both crossings; unresolved trailing branches can make the width unavailable while the peak remains valid.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

cv.peak_info

```
cv.peak_info(guess_or_options=None, options=None, **kwargs)
```

Summarize one peak, its tangent-corrected current and its half-peak metrics.

Parameters

Parameter	Meaning and default
guess_or_options	Potential guess or options dictionary; None uses automatic selection.
options, **kwargs	Segment, tangent, peak and display controls.

Returns

CVAnalysisResult with peak and half-peak metrics, tangent-corrected half-height width and width status, plus a display table.

Limitations and behavior

- A failed width estimate is marked Unavailable with its reason in diagnostics. plot all and print all control detailed child diagnostics.

Example

```
peak = cv_obj.peak_info({"segment": 1, "plot": False})
peak.table
```

Selected options

Option	Default	Meaning
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.
peak prominence	None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
percent threshold	None	Percent threshold used in peak or tangent selection.
tangent min points	None	Minimum points for tangent fitting; when omitted, eCAT derives a minimum from the pre-peak data length.
tangent potential	None	Manual tangent anchor potential; when omitted, eCAT anchors from the resolved tangent region. In complex CV analyses, the plural alias 'tangent potentials' accepts per-CV values.
tangent range	'auto'	'auto' chooses a pre-peak baseline region from derivative/curvature behavior before the peak.
exact potential	None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
guess potential	None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
segment	None	CV segment to analyze.

Complete registered options: e.describe_options("cv.peak_info").

Equations and parameter definitions

$$i_{\mathrm{baseline}}(E) = a\,E + b$$

$$i_p = i(E_p) - i_{\mathrm{baseline}}(E_p)$$

Symbol	Meaning and units	Input or controlling option
E, i	Potential (V), current (A unless another signal axis is selected)	Selected data; segment / segments, x axis and y axis.
E_p, i_p	Peak potential (V), tangent-corrected peak current (A)	guess potential / exact potential; peak prominence; noise window / noise polyorder.
a, b	Baseline slope (A/V) and intercept (A)	tangent range / tangent potential / tangent min points.

Scope and limitations

The baseline-corrected peak current differs from the raw current at E_p. peak_info also returns the width and half-peak quantities defined below.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

cv.half_wave_potential

```
cv.half_wave_potential(guess_or_options=None, options=None, **kwargs)
```

Calculate the midpoint of two selected peak potentials and their absolute separation.

Parameters

Parameter	Meaning and default
guess_or_options	Potential guess, a pair of guesses, or an options dictionary; None uses automatic selection.
options, **kwargs	Segment pair, peak-selection and display options.

Returns

CVAnalysisResult with E(1/2), delta E, peak 1 and peak 2. Potential values are in V; .primary is E(1/2).

Limitations and behavior

- This is the arithmetic peak midpoint, not an independent determination of a thermodynamic formal potential. Select a scientifically matched forward/return pair; both peaks must resolve.

Full options: e.describe_options("cv.half_wave_potential").

Example

```
wave = cv_obj.half_wave_potential({"segments": [1, 2], "plot": False})
midpoint_V = wave["E(1/2)"]
```

Selected options

Option	Default	Meaning
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.

Option	Default	Meaning
peak prominence	None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
percent threshold	None	Percent threshold used in peak or tangent selection.
tangent min points	None	Minimum points for tangent fitting; when omitted, eCAT derives a minimum from the pre-peak data length.
tangent potential	None	Manual tangent anchor potential; when omitted, eCAT anchors from the resolved tangent region. In complex CV analyses, the plural alias 'tangent potentials' accepts per-CV values.
tangent range	'auto'	'auto' chooses a pre-peak baseline region from derivative/curvature behavior before the peak.
exact potential	None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
guess potential	None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
segment	None	CV segment to analyze.

Complete registered options: e.describe_options("cv.half_wave_potential").

Equations and parameter definitions

$$E_{1/2} = \frac{E_{p1} + E_{p2}}{2}$$

$$\Delta\ E_p = |E_{p1} - E_{p2}|;\ \ r_i = |\frac{i_{pa}}{i_{pc}}|$$

Symbol	Meaning and units	Input or controlling option
E_{p1}, E_{p2}	Paired peak potentials (V)	segments and peak-selection options; peaks must belong to the intended wave.
i_{pa}, i_{pc}	Tangent-corrected anodic and cathodic peak currents (A)	peak-current baseline options, applied to the paired branches.

Scope and limitations

This midpoint is an operational CV peak midpoint. It need not equal a formal potential or a catalytic half-wave potential. wave_info additionally reports branch peak widths and half-peak shifts.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

cv.wave_info

```
cv.wave_info(guess_or_options=None, options=None, **kwargs)
```

Combine paired-wave potentials with tangent-corrected peak metrics.

Parameters

Parameter	Meaning and default
guess_or_options	Potential guess, paired guesses or an options dictionary; optional.
options, **kwargs	Paired segment selection, peak/tangent and display controls.

Returns

CVAnalysisResult with wave-level values, per-peak information, current ratio and a display table.

Limitations and behavior

- Requires one increasing-potential and one decreasing-potential branch. Peak-current ratios depend on the baseline and valid peak selection; they do not establish chemical reversibility by themselves.

Example

```
wave = cv_obj.wave_info({"segments": [1, 2], "plot": False})
wave.table
```

Selected options

Option	Default	Meaning
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.
peak prominence	None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
percent threshold	None	Percent threshold used in peak or tangent selection.
tangent min points	None	Minimum points for tangent fitting; when omitted, eCAT derives a minimum from the pre-peak data length.
tangent potential	None	Manual tangent anchor potential; when omitted, eCAT anchors from the resolved tangent region. In complex CV analyses, the plural alias 'tangent potentials' accepts per-CV values.
tangent range	'auto'	'auto' chooses a pre-peak baseline region from derivative/curvature behavior before the peak.
exact potential	None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.

Option	Default	Meaning
guess potential	None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
segment	None	CV segment to analyze.

Complete registered options: e.describe_options("cv.wave_info").

Equations and parameter definitions

$$E_{1/2} = \frac{E_{p1} + E_{p2}}{2}$$

$$\Delta\ E_p = |E_{p1} - E_{p2}|;\ \ r_i = |\frac{i_{pa}}{i_{pc}}|$$

Symbol	Meaning and units	Input or controlling option
E_{p1}, E_{p2}	Paired peak potentials (V)	segments and peak-selection options; peaks must belong to the intended wave.
i_{pa}, i_{pc}	Tangent-corrected anodic and cathodic peak currents (A)	peak-current baseline options, applied to the paired branches.

Scope and limitations

This midpoint is an operational CV peak midpoint. It need not equal a formal potential or a catalytic half-wave potential. wave_info additionally reports branch peak widths and half-peak shifts.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

trim

```
e.trim(cvs, window_or_options=None, options=None, **kwargs)
```

Select a potential window while preserving object/collection shape.

Parameters

Parameter	Meaning and default
cvs	One CV, a list, or nested CV groups.
window_or_options	Two-value potential window or trim-options dictionary.
options, **kwargs	Additional trim options; mode controls expand, pointwise or strict handling.

Returns

Trimmed CV copy/copies by default. inplace=True intentionally changes the input.

Limitations and behavior

- cv.trim(potential_window=None, options=None, **kwargs) is the single-object form. Connected-window expansion can retain points outside the requested bounds. Pointwise trimming can remove intervening history.

Full options: e.describe_options("trim").

Example

```
trimmed = e.trim(cvs, [-1.2, -0.4])
one = cv_obj.trim([-1.2, -0.4])
```

Selected options

Option	Default	Meaning
mode	'expand'	Trim mode: expand preserves connected CV segments, pointwise keeps only points inside the requested window, and strict raises if the window would disconnect the CV. Choices: expand, pointwise, strict.
potential window	None	Two-value potential window used to trim CV data. The shorthand e.trim(cvs, [start, stop]) fills this option.
x axis	None	Optional x-axis used to evaluate the trim window; by default eCAT uses the CV potential axis, including reference shift when active.
inplace	False	When True, mutate the source CV instead of returning a trimmed copy.

Complete registered options: e.describe_options("trim").

cv.filter

```
cv.filter(options=None)
```

Apply an explicit signal filter to a CV copy.

Parameters

Parameter	Meaning and default
options	CVFilterOptions/dictionary; method and method-specific parameters.

Returns

Filtered CV copy with processing provenance.

Limitations and behavior

- Signal filtering is separate from e.filter(), which selects objects by metadata. Filtering can change peak shape; retain raw data.

Full options: e.describe_options("cv.filter").

Example

```
e.describe_options("cv.filter")
filtered = cv_obj.filter({"method": "savgol"})
```

Selected options

Option	Default	Meaning
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	Savitzky-Golay smoothing polynomial order.
noise window	'auto'	Savitzky-Golay smoothing window.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s. A scalar applies to all CVs; a sequence supplies one value per CV.
x unit	'auto'	Requested x-axis unit.

Complete registered options: e.describe_options("cv.filter").

cv.analysis_segment_data

```
cv.analysis_segment_data(options)
```

Extract selected CV sweep coordinates using one shared selection helper.

Parameters

Parameter	Meaning and default
options	Dictionary with segment or segments and axis selection; required argument.

Returns

(x, y) for the selected segment(s), preserving acquisition direction.

Limitations and behavior

- Public segment selection is 1-based. The lower-level find_segment(idx=... or potential=..., x=None, options=None) returns a 0-based segment index, slice and (start,end) positional bounds. A potential may occur on more than one branch.

Example

```
potential, current = cv_obj.analysis_segment_data({"segment": 3})
```

Selected options

Option	Default	Meaning
segment	None	One 1-based sweep.
segments	None	Multiple 1-based sweeps; preserve acquisition order.
x axis	Object default	Coordinate column.
y axis	Object default	Signal column.

cv.current_at_potential

```
cv.current_at_potential(potential, options=None)
```

Report current at a requested potential separately for the selected branches.

Parameters

Parameter	Meaning and default
potential	Potential in V on the selected axis.
options	Segment selection and plotting/printing controls.

Returns

CVAnalysisResult mapping segment numbers to (potential,current) or None, with a tidy table and primary convenience attribute.

Limitations and behavior

- Omitting segment selection reports all segments. Branches must not be merged solely because they share a potential.

Full options: e.describe_options("cv.current_at_potential").

Example

```
sample = cv_obj.current_at_potential(-0.8, {"segment": 3, "plot": False})
sample.table
```

Selected options

Option	Default	Meaning
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.
peak prominence	None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
exact potential	None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.

Option	Default	Meaning
guess potential	None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
segment	None	CV segment to analyze.
segments	None	One or more CV segments to analyze.
x unit	'auto'	'auto' auto scales from the selected CV data and stored units.
y unit	'auto'	'auto' auto scales from the selected CV data and stored units.
x axis	None	Column or axis used as x data.

Complete registered options: e.describe_options("cv.current_at_potential").

Equations and parameter definitions

$$j = \operatorname{argmin}_j |E_j - E^*|; \ \text{return} \ (E_j, i_j)$$

Symbol	Meaning and units	Input or controlling option
E^*	Requested potential (V)	potential argument.
E_j, i_j	Selected-branch measurements	segment / segments; x axis / y axis.

Scope and limitations

This helper selects the nearest measured sample; it does not interpolate. It returns None for a segment if the nearest point is more than three times |mean ΔE| from the request. Without a segment selection it reports every segment separately.

cv.plot_program

```
cv.plot_program(options=None)
```

Inspect potential versus time, including the optional initial hold.

Parameters

Parameter	Meaning and default
options	Plot controls; plot quiet time=True by default.

Returns

Matplotlib Axes for the potential program.

Limitations and behavior

- Uses recorded time when available; otherwise reconstructs time from scan rate. The displayed quiet hold uses negative time and does not add measurements.

Full options: e.describe_options("cv.plot_program").

Example

```
ax = cv_obj.plot_program({"plot quiet time": True})
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("cv.plot_program").

CA CP and DPV reference

ca.current_at_time

```
ca.current_at_time(time=None, options=None)
```

Interpolate current at one time.

Parameters

Parameter	Meaning and default
time	Time in seconds; required, or supply options["time"]. A dictionary may be passed as the first argument.
options	Current-source, optional baseline correction and display controls.

Returns

ChronoAnalysisResult with current, time, current source, units and optional axes.

Limitations and behavior

- Requests outside the measured time range raise ValueError.

Full options: e.describe_options("ca.current_at_time").

Example

```
sample = ca_obj.current_at_time(10.0, {"plot": False, "print": False})
current_A = sample["current"]
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("ca.current_at_time").

Equations and parameter definitions

i(t*) = linear interpolation of the current trace at t*.

Symbol	Meaning and units	Input or controlling option
t*	Requested time (s)	time argument or time option.
i	Selected current (A)	Raw current by default; corrected current / baseline correction options select the corrected trace.

Scope and limitations

Time must lie within the measured range. Interpolation is in time, not potential.

ca.average_current

```
ca.average_current(time_range=None, options=None)
```

Calculate a time-weighted mean, not an unweighted row average.

Parameters

Parameter	Meaning and default
time_range	Two endpoints in seconds; None uses the whole resolved trace.
options	Current-source, baseline and display options.

Returns

ChronoAnalysisResult with average current, resolved time range, current source, units and optional axes.

Limitations and behavior

- The mean is integrated current divided by window duration; interpolation supplies window endpoints.

Full options: e.describe_options("ca.average_current").

Example

```
average = ca_obj.average_current([10, 30], {"plot": False})
current_A = average["average current"]
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("ca.average_current").

Equations and parameter definitions

$$i_{\text{avg}} = \frac{\int_{t_a}^{t_b} i(t)\ dt}{t_b - t_a} \quad \text{(trapezoidal quadrature)}$$

Symbol	Meaning and units	Input or controlling option
t_a, t_b	Window endpoints (s)	time_range argument; time range or window option.
i	Selected current (A)	corrected current / baseline correction; boundary currents are interpolated.

Scope and limitations

This is a time-weighted mean, not the arithmetic mean of samples. The window must have positive duration.

ca.rate_at_time

```
ca.rate_at_time(time=None, options=None)
```

Convert instantaneous current to electron flow and optional molecular rate or TOF.

Parameters

Parameter	Meaning and default
time	Seconds or a two-time interval, respectively; same selection rules as the corresponding current helper.
options	num electrons, faradaic efficiency, catalyst moles or catalyst concentration and volume, plus current/display options.

Returns

ChronoAnalysisResult with current metrics and electron flow; molecular rate/TOF only when the necessary stoichiometry and catalyst quantity are supplied.

Limitations and behavior

- Current is not a product-specific rate unless the electron stoichiometry and faradaic efficiency are justified.

Full options: e.describe_options("ca.rate_at_time").

Example

```
rate = ca_obj.rate_at_time(10.0, {"plot": False})
rate.table
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.

Option	Default	Meaning
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("ca.rate_at_time").

Equations and parameter definitions

$$\text{electron flow} = \frac{i}{F}$$

$$\text{molecular rate} = \frac{|i|\,\text{FE}}{n\,F}$$

$$\text{TOF} = \frac{\text{molecular rate}}{N_{\text{cat}}}$$

Symbol	Meaning and units	Input or controlling option
i	Current or time-weighted average current (A)	time or time range; same current selection as the corresponding current helper.
FE	Faradaic efficiency (fraction)	faradaic efficiency; conversion and validation follow the rate options.
n	Electrons per product molecule	num electrons / electron stoich.
N_{cat}	Catalyst amount (mol)	catalyst moles, or resolved catalyst concentration and volume.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

Electron flow is signed; molecular rate and TOF use current magnitude. A molecular rate requires n, and TOF additionally requires catalyst amount. These conversions do not establish product identity or FE.

ca.average_rate

```
ca.average_rate(time_range=None, options=None)
```

Convert time-averaged current to electron flow and optional molecular rate or TOF.

Parameters

Parameter	Meaning and default
time_range	Seconds or a two-time interval, respectively; same selection rules as the corresponding current helper.
options	num electrons, faradaic efficiency, catalyst moles or catalyst concentration and volume, plus current/display options.

Returns

ChronoAnalysisResult with current metrics and electron flow; molecular rate/TOF only when the necessary stoichiometry and catalyst quantity are supplied.

Limitations and behavior

- Current is not a product-specific rate unless the electron stoichiometry and faradaic efficiency are justified.

Full options: e.describe_options("ca.average_rate").

Example

```
rate = ca_obj.average_rate([10, 30], {"plot": False})
rate.table
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.

Option	Default	Meaning
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("ca.average_rate").

Equations and parameter definitions

$$\text{electron flow} = \frac{i}{F}$$

$$\text{molecular rate} = \frac{|i|\,\text{FE}}{n\,F}$$

$$\text{TOF} = \frac{\text{molecular rate}}{N_{\text{cat}}}$$

Symbol	Meaning and units	Input or controlling option
i	Current or time-weighted average current (A)	time or time range; same current selection as the corresponding current helper.
FE	Faradaic efficiency (fraction)	faradaic efficiency; conversion and validation follow the rate options.
n	Electrons per product molecule	num electrons / electron stoich.
N_{cat}	Catalyst amount (mol)	catalyst moles, or resolved catalyst concentration and volume.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

Electron flow is signed; molecular rate and TOF use current magnitude. A molecular rate requires n, and TOF additionally requires catalyst amount. These conversions do not establish product identity or FE.

ca.charge

```
ca.charge(options=None, **mpl_kwargs)
```

Integrate current over time and retain raw and optional baseline-corrected charge.

Parameters

Parameter	Meaning and default
options	Charge target, baseline correction, plotting and display options.
**mpl_kwargs	Additional Matplotlib keywords.

Returns

ChronoAnalysisResult with cumulative charge trace, final charge, target information and optional axes; corrected values are additional fields.

Limitations and behavior

- Charge is signed and in coulombs. Baseline subtraction changes the scientific quantity; report the correction.

Full options: e.describe_options("ca.charge").

Example

```
charge = ca_obj.charge({"plot": False})
charge_C = charge["final charge"]
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("ca.charge").

Equations and parameter definitions

$$Q_j = \sum_{k=1}^{j} i_k \left(t_k - t_{k-1} \right); \; Q_0 = 0$$

Symbol	Meaning and units	Input or controlling option
Q	Cumulative signed charge (C)	Current and Time columns; baseline correction selects a corrected charge trace.
Q_{target}	Target charge (C)	target charge, or target moles × target electrons × F.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

The current implementation uses a right-endpoint sum, not trapezoidal quadrature. It assigns no charge to the initial sample. A baseline correction can change the charge materially.

ca.time_at_charge

```
ca.time_at_charge(charge=None, options=None)
```

Find the time at which integrated charge reaches a target.

Parameters

Parameter	Meaning and default
charge	Target charge in C, or an options dictionary. None requires a target option.
options	target charge, or target moles and target electrons; display controls.

Returns

ChronoAnalysisResult with time, display time, target charge and units. An out-of-range target raises ValueError.

Limitations and behavior

- The current interpolation assumes increasing cumulative charge. Decreasing or nonmonotonic signed-charge traces need independent checking; the implementation does not resolve multiple crossings.

Full options: e.describe_options("ca.time_at_charge").

Example

```
target = ca_obj.time_at_charge({"target charge": 0.001, "plot": False})
time_s = target["time"]
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.

Option	Default	Meaning
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("ca.time_at_charge").

Equations and parameter definitions

t* = linear interpolation of t against Q at Q_{target}

$$Q_{\text{target}} = N_{\text{target}}\, n\, F \quad \text{(when a mole target is supplied)}$$

Symbol	Meaning and units	Input or controlling option
Q	Cumulative charge (C)	Same right-endpoint sum as ca.charge; this helper uses the uncorrected charge trace.
N_{target}, n	Target amount (mol) and electrons per conversion	target moles / target electrons; alternatively pass charge or target charge in C.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

The implementation calls np.interp with charge as the coordinate. Reliable inversion requires increasing charge; decreasing or nonmonotonic charge is not a general first-crossing calculation. A range check alone does not establish monotonicity.

cp.get_cycles

```
cp.get_cycles(options=None)
```

Segment the potential trace using large slope changes.

Parameters

Parameter	Meaning and default
options	Pass a dictionary explicitly in this checkout, for example {}. slope peaks overrides the expected number of boundaries.

Returns

dict with t (reconstructed seconds), v (potential array), and seg_idxs (index arrays per detected half-step). It does not return a list of DataFrames.

Limitations and behavior

- Current implementation reconstructs time from sample_int and uses slope-ranked boundaries. Inspect segmentation before assigning physical cycles.
- Although the signature says options=None, get_cycles() currently dereferences options directly; pass {}. cycle_info() has the same requirement.

Full options: e.describe_options("cp.get_cycles").

Example

```
parts = cp_obj.get_cycles({})
segments = parts["seg_idxs"]
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("cp.get_cycles").

cp.cycle_info

```
cp.cycle_info(options=None)
```

Summarize durations, capacities, efficiencies, energies and potentials.

Parameters

Parameter	Meaning and default
options	Explicit dictionary containing cycle and physical-quantity settings.

Returns

DataFrame with Cycle, Duration (s), capacity (mA·h), energy (mWh), potential (V) and efficiency/relative-capacity columns.

Limitations and behavior

- plot_cycles(options=None, **mpl_kwargs) returns Axes; cycling_plot(options=None, **mpl_kwargs) returns (Figure, (capacity_axes,efficiency_axes)). These are different contracts.

- Capacity depends on the programmed current metadata and correct half-step pairing; a slope partition is not an independent measurement of current.

Full options: e.describe_options("cp.cycle_info").

Example

```
cycles = cp_obj.cycle_info({})
ax = cp_obj.plot_cycles({"cycles": [1, 2]})
fig, axes = cp_obj.cycling_plot({"cycles": [1, 2]})
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("cp.cycle_info").

Equations and parameter definitions

$$\text{Capacity} = |I| \Delta\ t \times \frac{1000}{3600}\ \text{(mAh)}$$

$$\text{Energy} = |I \int E(t)\, dt| \times \frac{1000}{3600}\ \text{(mWh)}$$

$$\text{Coulombic efficiency} = 100 \frac{Q_{\text{discharge}}}{Q_{\text{charge}}}$$

$$\text{Energy efficiency} = 100 \frac{W_{\text{discharge}}}{W_{\text{charge}}}$$

Symbol	Meaning and units	Input or controlling option
I	Applied segment current (A)	Object cathodic_current / anodic_current metadata.
E, Δt	Measured potential (V); segment duration (s)	get_cycles segmentation.
Q, W	Discharge/charge capacity and energy	percent capacity normalizes each capacity series to its own first finite nonzero value.

Scope and limitations

The calculation assumes constant current per segment. Energy uses trapezoidal potential integration. With percent capacity enabled, the current code computes coulombic efficiency from the separately normalized capacities; use absolute capacity mode for the physical charge ratio.

cp.plot_cycles

```
cp.plot_cycles(options=None, **mpl_kwargs)
```

Overlay selected chronopotentiometry cycle traces.

Parameters

Parameter	Meaning and default
options	Pass a dictionary explicitly. cycles: one cycle, list, or inclusive (start,end[,step]) tuple. segment: full, discharge, charge or both. x-axis: capacity or time.
**mpl_kwargs	Additional Matplotlib styling keywords.

Returns

Matplotlib Axes. Uses the active axes unless new plot=True.

Limitations and behavior

- Calling plot_cycles() without a dictionary currently fails despite the options=None signature. Segmentation inherits the assumptions of get_cycles(); validate the detected half-steps.

Full options: e.describe_options("cp.plot_cycles").

Example

```
ax = cp_obj.plot_cycles(
    {"cycles": [1, 2], "new plot": True, "x-axis": "time"}
)
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.

Option	Default	Meaning
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("cp.plot_cycles").

cp.cycling_plot

```
cp.cycling_plot(options=None, **mpl_kwargs)
```

Plot cycle capacity and efficiency on two vertical axes.

Parameters

Parameter	Meaning and default
options	Dictionary or None. cycles or max cycles selects cycles; capacity mode and efficiency mode select curves; m_a window adds a moving average.
**mpl_kwargs	Plot styling, including cap_color, eff_color and alpha.

Returns

(Figure, (capacity_axes, efficiency_axes)). Unlike the other CP helpers, this method normalizes options=None to a dictionary.

Limitations and behavior

- Capacities and efficiencies inherit current metadata and segmentation assumptions. A moving average changes presentation, not the underlying cycle measurements.

Full options: e.describe_options("cp.cycling_plot").

Example

```
fig, (capacity_ax, efficiency_ax) = cp_obj.cycling_plot({"cycles": [1, 2]})
```

Selected options

Option	Default	Meaning
derivative	0	Derivative order or mode used for plotted data.
integrate	False	Integrate the selected signal when supported.

Option	Default	Meaning
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length when smoothing is requested; None disables Savitzky-Golay smoothing.
cycles	None	Cycle selection for CP cycle plots: int, list of ints, (start, end), or (start, end, step).
plot segment	None	Segment to emphasize or plot.
plot segments	None	Segments to emphasize or plot.
ip0	None	Non-catalytic reference peak current.
reference cv	None	Single reference CV used for current normalization or scaling.
reference cvs	None	List of reference CVs used for current normalization or scaling.
reference index	0	Index of the reference CV used for current normalization or scaling.
scan rate	None	Scan rate in V/s used for animation or normalization helpers; plotting otherwise uses the object's stored scan_rate when needed.

Complete registered options: e.describe_options("cp.cycling_plot").

Equations and parameter definitions

$$\text{Capacity} = |I|\Delta\ t \times \frac{1000}{3600}\ \text{(mAh)}$$

$$\text{Energy} = |I \int E(t)\,dt| \times \frac{1000}{3600}\ \text{(mWh)}$$

$$\text{Coulombic efficiency} = 100\,\frac{Q_{\text{discharge}}}{Q_{\text{charge}}}$$

$$\text{Energy efficiency} = 100\,\frac{W_{\text{discharge}}}{W_{\text{charge}}}$$

Symbol	Meaning and units	Input or controlling option
I	Applied segment current (A)	Object cathodic_current / anodic_current metadata.
E, Δt	Measured potential (V); segment duration (s)	get_cycles segmentation.
Q, W	Discharge/charge capacity and energy	percent capacity normalizes each capacity series to its own first finite nonzero value.

Scope and limitations

The calculation assumes constant current per segment. Energy uses trapezoidal potential integration. With percent capacity enabled, the current code computes coulombic efficiency from the separately normalized capacities; use absolute capacity mode for the physical charge ratio.

dpv.peak_potential

```
dpv.peak_potential(guess_or_options=None, options=None, **kwargs)
```

Locate a DPV extremum using peak-selection controls.

Parameters

Parameter	Meaning and default
guess_or_options	Numeric potential guess or options dictionary; optional.
options, **kwargs	Additional peak options, if supported by the displayed signature.

Returns

Tuple (peak_potential, peak_index). This differs from the CVAnalysisResult returned by cv.peak_potential().

Limitations and behavior

- A pulse-derived DPV peak is not a CV forward/return pair.

Full options: e.describe_options("dpv.peak_potential").

Example

```
potential_V, index = dpv_obj.peak_potential(
    {"guess potential": -0.5, "plot": False}
)
```

Selected options

Option	Default	Meaning
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.
peak prominence	None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
exact potential	None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
guess potential	None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
x unit	'auto'	'auto' auto scales from the selected CV data and stored units.
y unit	'auto'	'auto' auto scales from the selected CV data and stored units.
x axis	None	Column or axis used as x data.
xlabel	None	Custom x-axis label.
y axis	None	Column or axis used as y data.

Option	Default	Meaning
ylabel	None	Custom y-axis label.
pretty print	True	Use rich table display when printing object lists or summaries. False uses plain-text output when print is True.

Complete registered options: e.describe_options("cv.peak_potential").

Equations and parameter definitions

E_p = E at the selected local peak or requested exact potential.

Symbol	Meaning and units	Input or controlling option
E, i	Potential (V), current (A unless another signal axis is selected)	Selected data; segment / segments, x axis and y axis.
Ep	Peak potential (V); extraction only, not tangent-corrected peak current.	guess potential / exact potential; peak prominence; noise window / noise polyorder.

Scope and limitations

Peak location is an extraction algorithm, not a kinetic fit. Direction, smoothing, prominence and search choices can change which peak is selected. DPV has no CV sweep-pair interpretation.

dpv.fit_overlapping_peaks

```
dpv.fit_overlapping_peaks(options=None)
```

Fit a linear baseline plus two Gaussian peak components.

Parameters

Parameter	Meaning and default
options	guess potentials, fit window, center window, sigma guess/min/max, maxfev and plot/print controls.

Returns

DataFrame of two components, sorted by potential, with potential, amplitude, sigma, baseline current and total fitted current. attrs retain baseline, fit parameters and covariance.

Limitations and behavior

- closely_spaced_peaks(options=None) is an alias. This is a two-Gaussian descriptive model, not a mechanistic deconvolution.
- Cathodic amplitudes remain negative unless current inversion is requested. Pulse metadata uses SI attributes and unit-scaled display values.
- This untyped helper has no dedicated describe_options table in the current checkout. Use the options documented here; an unrecognized describe_options name returns the menu.

Example

```
components = dpv_obj.fit_overlapping_peaks(
    {
        "guess potentials": [-0.4, -0.6],
        "plot": False,
    }
)
components.attrs["covariance"]
```

Selected options

Option	Default	Meaning
guess potentials	Automatic	Two peak-center guesses in V.
fit window	None	Potential interval; at least eight finite points must remain.
sigma guess	max(3ΔE, span/30)	Initial Gaussian width; clipped to the width bounds.
sigma min	max(ΔE/2, span/1000)	Lower Gaussian width bound.
sigma max	span	Upper Gaussian width bound.

Equations and parameter definitions

$$i(E) = b_0 + b_1 E + A_1 \exp\left[-\frac{(E - E_1)^2}{2\sigma_1^2}\right] + A_2 \exp\left[-\frac{(E - E_2)^2}{2\sigma_2^2}\right]$$

Symbol	Meaning and units	Input or controlling option
b_0, b_1	Linear background intercept and slope	Fitted nuisance parameters.
A_1, A_2	Gaussian amplitudes (signal units)	Fitted peak heights above the linear background.
E_1, E_2	Peak centers (V)	guess potentials supplies initial values.
σ_1, σ_2	Gaussian widths (V)	sigma guess / sigma min / sigma max; fit window selects points.

Scope and limitations

Exactly two Gaussian peaks plus a linear background are fitted. This is a descriptive decomposition, not proof of two species or a mechanistic DPV model.

Complex Data Analysis

fowa

```
e.fowa(cvs, options=None)
```

Performs foot-of-the-wave analysis for one CV or a list of CVs. Printed output separates shared settings from per-CV table columns and shows fit quality diagnostics.

Parameters

Parameter	Meaning
cvs	catalytic cv object or list of cv objects.

Parameter	Meaning
options (dict or FOWAOptions)	reference-current, redox-potential, fitting, mechanism, and plotting controls.

Key options

Option	Default / type	Purpose and choices
ip0	None float or list[float] or None	Manual non-catalytic peak current. If omitted, FOWA can extract ip0 from non-catalytic CV inputs.
non-catalytic cv(s)	None object or None	Single non-catalytic CV used to extract the reference current automatically.
redox mode	'half wave' str	Method used to resolve redox potential: half-wave, half-peak, or manual redox potential. Choices: half wave, half peak, manual.
redox potential(s)	None float or list[float] or None	Manual redox potential; required only when redox mode is manual. The plural alias 'redox potentials' accepts per-CV values.
fit basis	'x' str	Axis or quantity used to select the fit region. Choices: x, y.
fit range	[0.0, 0.2] list[float] or tuple[float, float]	Single FOWA fit window on the selected fit basis. Use [min, max] for one shared window, or one [min, max] window per CV where supported.
wave range(s)	None list[float] or tuple[float, float] or None	Manual catalytic-wave potential window. Use 'wave range' for one shared range or 'wave ranges' for one [min, max] range per catalytic CV; if omitted, FOWA selects each wave independently.
background correction	'tangent' str or None	Background correction method used before FOWA; tangent uses the auto/manual tangent-baseline controls. Choices: tangent, start current.
diagnostic y axis	'i/ip0' str	Y axis used for FOWA diagnostic multiplot output. Choices: i/ip0, current.
plot	True bool	Whether to plot the result.
plot all	False bool	Whether to show diagnostic or child plots.
plot fit	True bool	Whether to overlay fitted curves or lines.
legend mode	'auto' str	Legend mode for discrete or gradient color encodings. Choices: auto, colorbar, discrete.
colorbar height scale	1.0 float	Scale factor for colorbar height in gradient legends.
mechanism	"EC'" str	Electrocatalytic mechanism label or model choice.

Option	Default / type	Purpose and choices
catalyst electrons	1 float	Catalyst redox-wave electron count n_cat used in kinetic equations. Aliases: n_cat, ncat.
turnover electrons	1 float	Turnover electron count n_turn used in kinetic equations. Aliases: n_turn, nturn.
min r2	0.98 float	Minimum recommended R2 threshold.
min fit points	20 int	Minimum recommended number of fit points.
reaction direction	'auto' str	FOWA reaction polarity: auto infers oxidation from an increasing-potential selected wave and reduction from a decreasing one, independently of current sign. Select one monotonic sweep, or explicitly choose oxidation/reduction for a return sweep. Choices: auto, oxidation, reduction.

Manual redox mode: when 'redox mode' is 'manual', FOWA still reports Catalytic $E_{cat/2}$ and $E_{cat/2}$ - $E_{1/2}$ using the manual redox potential as $E_{1/2}$. Set 'ecat shift warning threshold' to False to suppress that status check without removing other fit-quality status values.

FOWA diagnostic segment display: by default, plot all diagnostic multiplots keep the full CV traces. Set 'plot segments' to an integer or list of integers to restrict only the diagnostic plotted traces; the FOWA analysis segment is still controlled separately by 'segment' or 'segments'.

FOWA plot side effects: plot=True creates fit/diagnostic Matplotlib figures, and plot all=True also creates diagnostic multiplots using diagnostic y axis, default i/i_p^0. Use diagnostic y axis="current" for raw-current diagnostics. These figures are side effects; the returned AnalysisResult stores the summary table in result.table.

- Default redox mode is 'half wave'. If you provide 'redox potential', also set 'redox mode': 'manual'; eCAT raises an option error instead of silently switching modes.

Returns

AnalysisResult summary table. Use result.table for the FOWA table; use result.table.columns, result.table.loc, result.table.attrs, or result.table['kobs'] for DataFrame operations. FOWA tables include a compact Status column; detailed warning text is kept in result.diagnostics['full results'] and result.table.attrs['warnings'].

FOWA printed summaries: with print=True, FOWA prints a vertical Field/Value summary table for shared settings and reference information, then the symbolic k_{obs} equation with definitions, before the main FOWA result table. It does not print a second equation with n values substituted. With pretty print=False, the same summary shape is printed as plain text.

- Set 'warnings': False to suppress Python warnings while preserving Status and hidden warning-detail metadata in the returned table.
- FOWA unit metadata includes entries such as i_p^0 = A, redox potentials = V, R2 = dimensionless, and k_{obs}/TOF_{max} = s^{-1}.

Analysis functions return different result containers. Use result.table for table-based AnalysisResult workflows; check result.figure, result.axes and result.figures when provided. FOWA figures are plotting side effects and are not reliably collected in those attributes. plateau_current returns AnalysisResult, despite its stale DataFrame docstring. See the result ownership table rather than assuming every function has an identical return shape.

Scatter-fit plot colors: fit_model, fit_rate, fit_peak_current, fit_peak_potential, sevcik_analysis, trumpet_analysis, and multi_scatterplot use matching point and fit-line colors by default. Pass 'fit color' to override the fit line explicitly.

- For functions that draw multiple fits, 'fit color' may be a list; 'fit colors' is accepted as the plural alias, and describe_options displays this as 'fit color(s)'. Colors are consumed in plotted fit order, and the last color repeats if the list is shorter than the number of fits. If both spellings are supplied, eCAT raises an option error.

Example

```
table = e.fowa(
    cvs,
    {
        "non-catalytic cv": blank_cv,
        "redox mode": "half wave",
        "fit basis": "y",
        "fit range": [1, 5],
        "plot all": True,
    },
)
```

- Peak-current fits support 'y mode' and 'y0' to fit raw, delta, negative delta, ratio, or fractional current responses before any y transform.

Equations and parameter definitions

$$x_F = \frac{1}{1+\exp\left[-\frac{s\,n\,F\left(E-E_{\text{ref}}\right)}{R\,T}\right]};\ \ s=+1 \text{ for oxidation}, -1 \text{ for reduction}$$

$$y = \frac{i}{i_p^0};\ \ y = m\,x_F + b$$

$$\text{EC}'\text{ apparent } k_{\text{obs}} = \left[\frac{0.4463\,n\,m}{\left(n'\right)^{\sigma}}\right]^2 \frac{n\,F\,v}{R\,T}$$

Symbol	Meaning and units	Input or controlling option
E, E_{ref}	Applied and reference potentials (V)	redox mode: half wave uses the non-catalytic reference midpoint; half peak uses the catalytic half-peak estimate; manual uses redox potential(s).
s	Reaction direction sign	reaction direction = auto, oxidation or reduction; auto uses the selected monotonic potential-sweep direction.
i_p^0	Non-catalytic reference peak current (A)	ip0 / non catalytic current / non-catalytic cv(s).
n, n′, σ	Catalyst electron count, turnover electron count, pathway exponent	catalyst electrons (num electrons fallback), turnover electrons, sigma.
m, b	Fitted slope and intercept	fit basis / fit range; min fit points / min r2; selected foot region.
v, T, k_{obs}	V/s, K, apparent rate s^{-1}	CV scan_rate and temperature; mechanism / custom formula select conversion.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

The displayed EC′ expression is mechanism-specific. The catalytic $E_{cat/2}$ and non-catalytic $E_{1/2}$ need not coincide. A custom formula replaces the rate conversion and must document its own symbols. Automatic direction assumes the reaction follows the selected sweep direction; choose an explicit direction for unusual return-sweep chemistry. The half peak option is not guaranteed to give plateau-derived $E_{cat/2}$. To use a measured $E_{cat/2}$, supply it with manual mode. Fitted line display stays within the measured transformed-x span and the observed signed normalized-current envelope expanded to include zero. It never exceeds the measured current magnitude; clipping does not change regression coefficients.

Method and further reading

Start with the catalytic mechanism and its appropriate FOWA expression, then choose the foot region and inspect the fit diagnostics. Wang and Johnson discuss the role of $E_{cat/2}$ and interpretation of multielectron waves. A catalytic half-wave potential is not automatically the non-catalytic redox midpoint; a fitted line alone does not establish the mechanism.

- Costentin, C.; Drouet, S.; Robert, M.; Savéant, J.-M. (2012). Turnover Numbers, Turnover Frequencies, and Overpotential in Molecular Catalysis of Electrochemical Reactions. Cyclic Voltammetry and Preparative-Scale Electrolysis. J. Am. Chem. Soc. 134, 11235–11242. https://doi.org/10.1021/ja303560c
- Wang, V. C.-C.; Johnson, B. A. (2019). Interpreting the Electrocatalytic Voltammetry of Homogeneous Catalysts by the Foot of the Wave Analysis and Its Wider Implications. ACS Catalysis 9, 7109–7123. https://doi.org/10.1021/acscatal.9b00850
- Rountree, E. S.; McCarthy, B. D.; Eisenhart, T. T.; Dempsey, J. L. (2014). Evaluation of Homogeneous Electrocatalysts by Cyclic Voltammetry. Inorg. Chem. 53, 9983–10002. https://doi.org/10.1021/ic500658x

fit_peak_current

```
e.fit_peak_current(cvs, options=None)
```

Fits peak current against scan rate or another resolved x quantity. Normalized CVs with an i/i_p^0 axis are supported. The fit uses the shared fit_model engine; use 'fit model' for linear, power, power offset, exponential, Michaelis-Menten, logistic, callable, or formula fits.

Parameters

Parameter	Meaning
cvs	list of CVs.
options (dict or FitPeakCurrentOptions)	peak extraction, x/y transformation, model, display and diagnostics controls.

Key options

Option	Default / type	Purpose and choices
segment	None int or None	CV segment to analyze.
guess potential(s)	None float or list[float] or list[list[float]] or None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
exact potential(s)	None float or list[float] or None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
tangent range	'auto' str or float or list[float] or tuple[float, float]	'auto' is passed to peak_current so peak-current extraction uses automatic tangent-baseline selection.
fit	True bool	Whether to fit the analysis result.
fit indices	None object or None	Row/position-based fit selection on the resolved points. Use [start, stop] with Python-style exclusive stop; when omitted, all resolved points are included.

Option	Default / type	Purpose and choices
x transform	'^0.5' str or float or int or None	Transform x before fitting. The configured default '^0.5' is square root: use 0.5, 'sqrt', or 'square root'; any numeric power (including negative values) is accepted. None or 'identity' uses raw x. This is separate from 'fit model': use 'fit model': 'power' with 'x transform': 'identity' to estimate a power-law exponent. xscale changes axis spacing only.
plot	True bool	Whether to plot the result.
plot all	False bool	Whether to show diagnostic or child plots.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
legend	False bool	Whether to show a plot legend.
x axis	None str or None	Column or axis used as x data.
species	None str or None	Species used when auto-resolving concentration-based x data from CV compounds/concentrations metadata.

Returns

ScatterFitResult, an AnalysisResult child, with table, fits, fit_table, figure, axes, and summary metadata. Use result.table and result.fits in new notebooks; tuple unpacking is not part of the API.

Example

```
result = e.fit_peak_current(
    cvs, {"segment": 1, "guess potential": -1.5, "plot all": True}
)
```

- Y adjustment options: 'y mode' may be raw, delta, negative delta, ratio, or fractional; 'y0' may be a scalar or mapping. The adjustment is applied before any y transform and before fitting. Plotted y-axis labels show the y-mode expression first and then wrap it with any y transform, matching the order used for fitting. Baseline labels use a superscript 0, for example k_{obs}^{0}. When the input current axis is normalized i/i_p^{0}, fit_peak_current labels the fitted peak-current axis as i_p/i_p^{0}.
- For plot-all diagnostics, use 'plot segment' or 'plot segments' to limit the displayed CV traces without changing the analysis 'segment'/'segments'.
- fit_peak_current can infer a varying mole-fraction token such as 0.8xD2O even when the same species also appears at a fixed molar concentration, such as 2.8MD2O.

Choosing the x transform

Option	Meaning
x axis	Choose scan rate, concentration or the supported resolved quantity.
x transform	Transform coordinates before fitting. For example 0.5 selects a square-root power; use describe_options for named transforms.
fit model	Model fitted in the transformed coordinates. A linear fit after a transform is not the same as fitting every named model to raw data.

Option	Meaning
xscale / plot scale	Matplotlib axis spacing where supported; it does not replace a data transform.

```
result = e.fit_peak_current(
    cvs,
    {
        "x axis": "scan rate",
        "x transform": 0.5,
        "fit model": "linear",
        "plot all": True,
    },
)
```

Equations and parameter definitions

linear: $y = m\,x + b$

power: $y = A\,x^{p}$; power_offset: $y = b + A\,x^{p}$

exponential: $y = b + A\exp(k\,x)$

michaelis_menten: $y = \dfrac{V_{\max}\,x}{K_m + x}$

logistic: $y = b + \dfrac{L}{1 + \exp[-k(x - x_0)]}$

For transformed peak/rate fits: $x_{fit} = T_x(x)$, $y_{fit} = T_y(y)$; fit the model to these coordinates.

Symbol	Meaning and units	Input or controlling option
x, y	Coordinates in the selected units	x column / y column for fit_model; x axis / species for peak fits; x column / metric for fit_rate.
m, b, A, p, k, V_{max}, K_m, L, x_0	Fitted coefficients; units follow the chosen coordinates	fit model; fit init / fit bounds / fit params.
r, σ	Residual and supplied uncertainty	fit residual; fit sigma / fit absolute sigma; fit indices selects observations.
T_x, T_y	Transformations before fitting	x transform / y transform in peak/rate wrappers: identity, powers, sqrt, log10, ln or callable; transform mode and floors govern nonpositive inputs.

Scope and limitations

fit_model fits its supplied coordinates directly and does not apply the peak/rate transformation layer. A plot-axis scale does not transform a fit. Coefficient units depend on the transformed coordinates; a descriptive regression is not a mechanistic rate law.

Method and further reading

Use the electrochemical model to select the x transform; a generic power fit is descriptive unless its physical assumptions are justified.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

fit_peak_potential

```
e.fit_peak_potential(cvs, options=None)
```

Fits peak potential versus log10(scan rate) or log10(concentration). Non-positive x values are retained in the returned table but excluded from log-transformed plotting and fitting. The fit uses the shared fit_model engine; use 'fit model' for direct model fits on the resolved x/y points.

Parameters

Parameter	Meaning
cvs	list of CVs.
options (dict or FitPeakPotentialOptions)	peak-potential extraction, automatic log10 x coordinates, model and display controls. A user x transform is not accepted by this option model.

Key options

Option	Default / type	Purpose and choices
segment	None int or None	CV segment to analyze.
guess potential(s)	None float or list[float] or list[list[float]] or None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
exact potential(s)	None float or list[float] or None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
fit	True bool	Whether to fit the analysis result.
fit indices	None object or None	Row/position-based fit selection on the resolved points. Use [start, stop] with Python-style exclusive stop; when omitted, all resolved points are included.
plot	True bool	Whether to plot the result.
plot all	False bool	Whether to show diagnostic or child plots.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
legend	False bool	Whether to show a plot legend.
x axis	None str or None	Column or axis used as x data.
species	None str or None	Species used when auto-resolving concentration-based x data from CV compounds/concentrations metadata.

Returns

ScatterFitResult, an AnalysisResult child, with table, fits, fit_table, figure, axes, and summary metadata. Use result.table and result.fits in new notebooks; tuple unpacking is not part of the API.

Example

```
result = e.fit_peak_potential(cvs, {"segment": 1, "guess potential": -0.4})
```

- Rate fits support 'y mode' and 'y0' to fit raw, delta, negative delta, ratio, or fractional metric responses before any y transform. Plotted y-axis labels show the y-mode expression first and then wrap it with any y transform, matching the order used for fitting. Baseline labels use a superscript 0, for example k_{obs}^{0}.

Equations and parameter definitions

linear: $y = m\,x + b$

power: $y = A\,x^{p}$; power_offset: $y = b + A\,x^{p}$

exponential: $y = b + A\exp(k\,x)$

michaelis_menten: $y = \dfrac{V_{max}\,x}{K_m + x}$

logistic: $y = b + \dfrac{L}{1 + \exp[-k(x - x_0)]}$

For transformed peak/rate fits: $x_{fit} = T_x(x)$, $y_{fit} = T_y(y)$; fit the model to these coordinates.

Symbol	Meaning and units	Input or controlling option
x, y	Coordinates in the selected units	x column / y column for fit_model; x axis / species for peak fits; x column / metric for fit_rate.
m, b, A, p, k, V_{max}, K_m, L, x_0	Fitted coefficients; units follow the chosen coordinates	fit model; fit init / fit bounds / fit params.
r, σ	Residual and supplied uncertainty	fit residual; fit sigma / fit absolute sigma; fit indices selects observations.
T_x, T_y	Transformations before fitting	x transform / y transform in peak/rate wrappers: identity, powers, sqrt, log10, ln or callable; transform mode and floors govern nonpositive inputs.

Scope and limitations

fit_model fits its supplied coordinates directly and does not apply the peak/rate transformation layer. A plot-axis scale does not transform a fit. Coefficient units depend on the transformed coordinates; a descriptive regression is not a mechanistic rate law.

Method and further reading

Peak-shift trends can be reported descriptively. Laviron kinetic interpretation specifically assumes the relevant diffusionless model.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361
- Laviron, E. (1979). General expression of the linear potential sweep voltammogram in the case of diffusionless electrochemical systems. J. Electroanal. Chem. Interfacial Electrochem. 101, 19–28. https://doi.org/10.1016/S0022-0728(79)80075-3

fit_model

```
e.fit_model(x_or_result, y=None, model=None, options=None)
```

Fit a named model (linear, power, power offset, exponential, Michaelis–Menten or logistic), callable or restricted formula to arrays or result-table columns. Use fit model, fit init, fit bounds, fit residual and fit max evals in wrapper functions. Standalone fit_model also accepts the corresponding short aliases model, init, bounds, residual and indices.

- Parameters are fitted to selected rows; predictions and residuals remain aligned to original rows. Use fit indices for row selection and fit line range only for the displayed line extent. The older fit range/fit ranges instructions do not apply to these scatter fitters in this checkout. FOWA has a separate fit range option with different semantics.

Parameters

Parameter	Meaning
x_or_result	x array, DataFrame, AnalysisResult or ScatterFitResult.
y	y array when x_or_result is an x array; otherwise None.
model	Named model, callable or restricted formula; None uses options/defaults.
options	Column selection, initialization, bounds, residual objective, fit windows and display.

Returns

ScatterFitResult with table, fit_table, fit_model_results, diagnostics and optional figure/axes.

```
result = e.fit_model(
    [1, 2, 3],
    [2, 4, 6],
    model="linear",
    options={"plot": False, "print": False},
)
result.fit_table
```

Selected options

Option	Default	Meaning
fit	True	Whether to fit the analysis result.
fit indices	None	Row/position-based fit selection. Use [start, stop] with Python-style exclusive stop, None for an open-ended side, a boolean mask, explicit indices, multiple windows for one disconnected fit, or a dict for named fits.
x column	'auto'	DataFrame column used as x data.
y column	'auto'	DataFrame column used as y data.
pretty print	True	Use rich table display when printing object lists or summaries. False uses plain-text output when print is True.
print	True	Whether to emit output. False suppresses output; it is independent of pretty print.
sig figs	4	Significant figures used for reported values.
curve fit options	None	Advanced scipy.optimize.curve_fit keyword passthrough; eCAT still owns fit init and bounds.

Option	Default	Meaning
fit absolute sigma	False	Whether fit sigma is absolute for covariance/error estimates.
fit bounds	'auto'	Lower and upper bounds for a fit model; use None for an unbounded side.
fit check finite	None	check_finite argument passed to scipy.optimize.curve_fit.
fit init	'auto'	Initial parameter guesses for a fit model.

Complete registered options: e.describe_options("fit_model").

Equations and parameter definitions

linear: $y = m\,x + b$

power: $y = A\,x^{p}$; power_offset: $y = b + A\,x^{p}$

exponential: $y = b + A\exp(k\,x)$

michaelis_menten: $y = \dfrac{V_{\max}\,x}{K_m + x}$

logistic: $y = b + \dfrac{L}{1 + \exp[-k(x - x_0)]}$

For transformed peak/rate fits: $x_{fit} = T_x(x)$, $y_{fit} = T_y(y)$; fit the model to these coordinates.

Symbol	Meaning and units	Input or controlling option
x, y	Coordinates in the selected units	x column / y column for fit_model; x axis / species for peak fits; x column / metric for fit_rate.
m, b, A, p, k, V_{max}, K_m, L, x_0	Fitted coefficients; units follow the chosen coordinates	fit model; fit init / fit bounds / fit params.
r, σ	Residual and supplied uncertainty	fit residual; fit sigma / fit absolute sigma; fit indices selects observations.
T_x, T_y	Transformations before fitting	x transform / y transform in peak/rate wrappers: identity, powers, sqrt, log10, ln or callable; transform mode and floors govern nonpositive inputs.

Scope and limitations

fit_model fits its supplied coordinates directly and does not apply the peak/rate transformation layer. A plot-axis scale does not transform a fit. Coefficient units depend on the transformed coordinates; a descriptive regression is not a mechanistic rate law.

fit_rate

```
e.fit_rate(df, options=None)
```

Fits a FOWA-derived metric such as k_{obs} against a concentration, scan-rate, or transformed x column. When input DataFrames provide unit metadata, k_{obs} plots include units such as s^{-1} in the y-axis label.

Parameters

Parameter	Meaning
df	FOWA AnalysisResult, FOWA output table, or compatible DataFrame.

- Use fit indices for row selection: [start, stop] includes start and excludes stop, with None for an open end. Nested windows select disconnected rows for one fit; a dictionary supplies separate named fits. fit line range only changes the displayed line extent. Choose fit model and its initialization, bounds and residual settings explicitly.
- fit_rate can infer mole-fraction x values from FOWA-style Compounds or Plot Label columns and labels them as χ(species).
- Result tables produced by FOWA and rate-style analyses use table.attrs['units'] for derived column units where available.
- options (dict or FitRateOptions): metric, x selection, filtering, fit, and plotting controls.

Key options

Option	Default / type	Purpose and choices
metric	'kobs' str	Metric column to fit; if omitted from plotting helpers, common rate/output metric columns are preferred.
x column	'auto' str or None	'auto' auto chooses a sensible x column: varying scan rate first, then concentration columns/compound metadata.
species	None str or None	Species used to filter concentration-column auto-resolution when x column is auto.
x transform	None str or float or int or None	Transform applied to x values.
plot log log	False bool	Whether to include a log-log plot.
exclude warnings	False bool	Exclude rows or fits that emitted analysis warnings.
exclude low r2	False bool	Exclude fits whose R2 is below the requested threshold.
fit indices	None object or None	Row/position-based fit selection on the resolved rate table. Use [start, stop], disconnected windows, or a dict of named fits; stops follow Python's exclusive convention.
fit	True bool	Whether to fit the analysis result.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
plot	True bool	Whether to plot the result.
legend	False bool	Whether to show a plot legend.
local slope mode	'adjacent' str	Method for calculating local slopes. Choices: adjacent, gradient.

Option	Default / type	Purpose and choices
print local slopes	False bool	Whether to print local slope diagnostics.

Returns

ScatterFitResult, an AnalysisResult child, with table, fits, fit_table, figure, axes, and summary metadata. Use result.table and result.fits; tuple unpacking is not part of the API.

Example

```
rate = e.fit_rate(
    fowa_result,
    {
        "metric": "kobs",
        "transform mode": "log-log",
        "fit indices": {"early": [0, 3], "full": [0, None]},
    },
)
```

- For example, {'y mode': 'fractional'} fits y/y0 - 1, where y0 defaults to the first valid raw y value in each series.

Equations and parameter definitions

linear: $y = m\,x + b$

power: $y = A\,x^{p}$; power_offset: $y = b + A\,x^{p}$

exponential: $y = b + A\exp(k\,x)$

michaelis_menten: $y = \dfrac{V_{\max}\,x}{K_m + x}$

logistic: $y = b + \dfrac{L}{1 + \exp[-k(x - x_0)]}$

For transformed peak/rate fits: $x_{fit} = T_x(x)$, $y_{fit} = T_y(y)$; fit the model to these coordinates.

Symbol	Meaning and units	Input or controlling option
x, y	Coordinates in the selected units	x column / y column for fit_model; x axis / species for peak fits; x column / metric for fit_rate.
m, b, A, p, k, V_{max}, K_m, L, x_0	Fitted coefficients; units follow the chosen coordinates	fit model; fit init / fit bounds / fit params.
r, σ	Residual and supplied uncertainty	fit residual; fit sigma / fit absolute sigma; fit indices selects observations.
T_x, T_y	Transformations before fitting	x transform / y transform in peak/rate wrappers: identity, powers, sqrt, log10, ln or callable; transform mode and floors govern nonpositive inputs.

Scope and limitations

fit_model fits its supplied coordinates directly and does not apply the peak/rate transformation layer. A plot-axis scale does not transform a fit. Coefficient units depend on the transformed coordinates; a descriptive regression is not a mechanistic rate law.

Method and further reading

Use for interpreting catalytic rate quantities supplied to this fitter. The paper does not validate every generic regression model available in eCAT.

- Rountree, E. S.; McCarthy, B. D.; Eisenhart, T. T.; Dempsey, J. L. (2014). Evaluation of Homogeneous Electrocatalysts by Cyclic Voltammetry. Inorg. Chem. 53, 9983–10002. https://doi.org/10.1021/ic500658x

sevcik_analysis

```
e.sevcik_analysis(cvs, options=None)
```

Runs the bulk diffusion-controlled Sevcik peak-current analysis over CVs. Scan-rate series always fit peak current against the square root of scan rate; use surface_coverage_analysis() for surface-confined peaks that are linear in scan rate.

Parameters

Parameter	Meaning
cvs	list of CVs.
options (dict or SevcikAnalysisOptions)	peak-current, scan-rate, fit, and plot controls.

Key options

Option	Default / type	Purpose and choices
segment	None int or None	CV segment to analyze.
guess potential(s)	None float or list[float] or list[list[float]] or None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
tangent range	'auto' str or float or list[float] or tuple[float, float]	'auto' is passed to peak_current so peak-current extraction uses automatic tangent-baseline selection.
fit indices	None object or None	Row/position-based fit selection on the resolved points. Use [start, stop] with Python-style exclusive stop; when omitted, all resolved points are included.
plot	True bool	Whether to plot the result.
plot all	False bool	Whether to show diagnostic or child plots.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
legend	False bool	Whether to show a plot legend.

Returns

ScatterFitResult, an AnalysisResult child. When plot=True, Sevcik-style helpers create/update Matplotlib figures as a side effect; the fit result remains the returned analysis object.

Example

```
result = e.sevcik_analysis(
    cvs, {"segment": 1, "guess potential": -1.2, "plot": True}
)
```

Equations and parameter definitions

$$|i_p| = 0.4463\, n\, F\, S\, C \sqrt{\frac{n\, F\, D\, v}{R\, T}}$$

Scan-rate series: $D = \frac{R\, T}{(n\, F)^3} \left[\frac{m}{0.4463\, S\, C}\right]^2$

Concentration series: $D = \frac{R\, T}{(n\, F)^3\, v\, S^2} \left[\frac{m_C}{0.4463}\right]^2$

Symbol	Meaning and units	Input or controlling option
m, m_c	Slope of i_p vs sqrt(v), or i_p vs C	fit indices chooses points. Slopes are converted to A and mol/cm³ before calculating D.
n	Electron count	num electrons.
S, T	Area (cm²), temperature (K)	First CV electrode_area and temperature metadata.
C	Selected species concentration (mol/cm³)	C / species or metadata; mol/L is divided by 1000.
v, D	Scan rate (V/s); derived diffusion coefficient (cm²/s)	CV scan_rate metadata; D is the result.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

Assumes reversible electron transfer with planar semi-infinite diffusion. Use a fixed concentration for a scan-rate series, or a fixed scan rate for a concentration series. A nonzero intercept or nonlinear trend can signal background or model failure.

Method and further reading

Read for diffusion-controlled peak-current scaling with the square root of scan rate. Linearity alone does not establish all Randles–Ševčík assumptions.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

reversibility_analysis

```
e.reversibility_analysis(cvs, options=None)
```

Assesses electron-transfer reversibility and chemical return-current behavior for one chemical condition measured at three or more distinct scan rates. Replicates remain in the result table while trend fits use scan-rate means. Five or more rates are recommended.

Parameters

Parameter	Meaning
cvs	one CV scan-rate series; group mixed conditions first with e.group(...).
options	phase, segment/peak controls, n, D, C/species, area, temperature, agreement tolerance, print, and plot controls.

Decision tree

- Bulk: convert eligible n ΔEp to Nicholson ψ and Λ; Matsuda–Ayabe uses Λ ≥ 15 for reversible and $10^{-2(1+\alpha)} < \Lambda < 15$ for quasi-reversible. Nicholson k^0 is used only for $0.1 \le \psi \le 7$.
- If D is omitted, exact species concentration plus area and temperature allow branch-specific Sevcik Dapp estimates. Both branches must meet min R2 and agreement tolerance.
- A series that remains reversible reports a k^0 lower bound. Irreversible behavior is only reported when large peak separation is corroborated by both E_p-$E_{p/2}$ and E_p-v asymptotes.
- Surface: require i_p linear in scan rate and compare Delta E_p with max(configured tolerance, three median potential increments). Laviron is eligible only for at least two n Delta E_p > 200 mV points.
- Chemical conclusion: return/forward peak-current ratios within current ratio tolerance of unity are chemically reversible over the observed timescale; persistent deviations or recovery toward unity at faster scans indicate coupled chemistry. Sparse or conflicting evidence is indeterminate.

Returns

AnalysisResult with per-CV table, scan-rate means, exact decision evidence, kinetic estimates or bounds, warnings, and diagnostic figures.

Example

```
result = e.reversibility_analysis(
    cvs,
    {
        "phase": "bulk",
        "guess potential": -1.0,
        "num electrons": 1,
        "D": 1e-5,
    },
)
```

Selected options

Option	Default	Meaning
phase	'bulk'	Physical model for the series. Bulk is the default diffusion-controlled model; use surface explicitly for adsorbed or surface-confined couples. Choices: bulk, surface.
psi source	'agarwal table'	Nicholson conversion: Agarwal (2025) Table 4 by default; choose lavagnini or empirical for Lavagnini et al. (2004). Choices: agarwal table, empirical.
num electrons	1	Number of electrons in the redox event.
alpha	0.5	Electron-transfer coefficient used for kinetic boundaries or fitting.
D	None	Bulk diffusion coefficient in cm^2/s. If omitted, eCAT attempts branch-specific Sevcik Dapp estimates when area and concentration are available.

Option	Default	Meaning
C	None	Bulk concentration in mol/cm^3. If omitted, exact species metadata may be used when species is provided.
species	None	Chemical species used to resolve concentration or metadata. For normalize, exact-matches cv.compounds and pulls the paired cv.concentrations value when C/C unit are omitted.
current ratio tolerance	0.1	Maximum absolute deviation of tangent-corrected |ipa/ipc| from unity for chemical-reversibility evidence. Independent of agreement tolerance and configurable globally with set_defaults.
agreement tolerance	0.25	Maximum symmetric fractional disagreement for independent estimates of the same quantity. This practical threshold can be changed globally with set_defaults.
peak separation tolerance	0.01	Practical zero-separation tolerance in V for surface-confined couples; the effective value is at least three median potential increments.
min r2	0.98	Minimum R2 for treating a regression as method-eligible evidence.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.

Complete registered options: e.describe_options("reversibility_analysis").

Equations and parameter definitions

$$z = 1000\, n\, |E_{pa} - E_{pc}| \ \text{(mV)}$$

Default: ψ = linear interpolation of Agarwal (2025), Table 4, at z.

Optional Lavagnini: $\psi = \dfrac{-0.6288 + 0.0021\, z}{1 - 0.017\, z}$

Bulk: $\Lambda = \sqrt{\pi}\,\psi;\ \ k^0 = \Lambda \sqrt{\dfrac{D\, n\, F\, v}{R\, T}}$

Matsuda–Ayabe: reversible $\Lambda \geq 15$
Quasi-reversible $10^{-2(1+\alpha)} < \Lambda < 15$
Irreversible $\Lambda \leq 10^{-2(1+\alpha)}$

Chemical recovery: $r_i = \left|\dfrac{i_{pa}}{i_{pc}}\right|;\ \ d_i = |r_i - 1|;\ \ \text{fit } d_i \text{ against } \log_{10}(v)$

Surface: $i_p = \dfrac{n^2 F^2 S \Gamma\ \ v}{4\, R\, T}$

Effective ΔEp tolerance = max(configured tolerance, 3 × median potential increment).

Surface Laviron region: $n\Delta\ \ E_p > 200\ \text{mV};\ \ E_{pa} = m_a \ln(v) + b_a;\ \ E_{pc} = m_c \ln(v) + b_c$

$$\alpha = \frac{R\, T}{n\, F\, m_a};\ \ \beta = -\frac{R\, T}{n\, F\, m_c}$$

$$A = \frac{R\, T}{\alpha\ \ n\, F};\ \ B = \frac{R\, T}{\beta\ \ n\, F}$$

$$\ln(k_s) = \frac{A\ln\left(\frac{\alpha\ nF}{RT}\right) + B\ln\left(\frac{\beta\ nF}{RT}\right) - (b_a - b_c)}{A + B}$$

Symbol	Meaning and units	Input or controlling option
ψ, z	Dimensionless kinetic parameter; scaled peak separation (mV)	psi source = agarwal table (default), or lavagnini / empirical.
E_{pa}, E_{pc}, n	Paired peak potentials (V), electron count	segment / segments and peak-selection options; num electrons.
phase	Bulk diffusion or surface-confined model	phase = bulk or surface; the ψ source affects bulk analysis only.
α, β	Transfer coefficients (dimensionless)	Bulk alpha option sets the Matsuda–Ayabe boundary; surface α/β are independently inferred from slopes.
D, S, C, T, v	cm^2/s, cm^2, mol/cm^3, K, V/s	D or eligible Ševčík estimate; electrode area, C / species, temperature or shared metadata; CV scan_rate.
r_i, d_i	Current-ratio recovery evidence	current ratio tolerance; min r2 controls trend acceptance.
Γ, k_s	Surface coverage (mol/cm^2); surface rate (s^{-1})	Surface model; ks is distinct from bulk k0 in cm/s.
m_a, m_c, b_a, b_c, A, B	Surface slopes/intercepts and derived voltage scales	High-separation paired branch fits; at least two eligible scan rates.
Model agreement	Relative consistency of independent estimates	agreement tolerance; peak separation tolerance and min r2 control the surface assessment.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

Both functions now share the same ψ converter and default. Reversibility adds its own eligibility gate (0.1 ≤ ψ ≤ 7 and z ≤ 212 mV). Values below 61 mV are treated as too reversible for a finite Nicholson rate, not assigned a measured infinite rate. Chemical recovery requires at least three finite ratios: all deviations within tolerance support recovery; mean deviation above tolerance or a reliable decrease toward unity flags coupled chemistry. These are eCAT decision rules, not proof of a unique reaction mechanism. Surface α and β are checked individually; the code does not enforce α+β=1.

Method and further reading

The bulk model combines Matsuda–Ayabe regions with the same selectable Nicholson conversion as nicholson_analysis (Agarwal by default; Lavagnini optional). Surface analysis uses the Laviron framework. Chemical recovery is assessed separately through current ratios; its numerical acceptance tolerances are eCAT settings, not universal literature constants.

- Matsuda, H.; Ayabe, Y. (1955). Zur Theorie der Randles-Sevčikschen Kathodenstrahl-Polarographie. Zeitschrift für Elektrochemie 59, 494–503. https://doi.org/10.1002/bbpc.19550590605
- Nicholson, R. S. (1965). Theory and Application of Cyclic Voltammetry for Measurement of Electrode Reaction Kinetics. Anal. Chem. 37, 1351–1355. https://doi.org/10.1021/ac60230a016
- Agarwal, R. (2025). The Nicholson Method of Determination of the Standard Rate Constant of a Quasireversible Redox Couple Employing Cyclic Voltammetry: Everything One Needs to Know! ACS Electrochemistry 1, 1885–1894. https://doi.org/10.1021/acselectrochem.5c00214

- Lavagnini, I.; Antiochia, R.; Magno, F. (2004). An Extended Method for the Practical Evaluation of the Standard Rate Constant from Cyclic Voltammetric Data. Electroanalysis 16, 505–506. https://doi.org/10.1002/elan.200302851
- Laviron, E. (1979). General expression of the linear potential sweep voltammogram in the case of diffusionless electrochemical systems. J. Electroanal. Chem. Interfacial Electrochem. 101, 19–28. https://doi.org/10.1016/S0022-0728(79)80075-3
- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

Replicates remain individually visible. For each scan rate, eCAT first averages the measured peak separation and then applies the selected Nicholson conversion to that mean. Per-scan eligibility flags remain available; out-of-range repeats are not silently removed from the mean. The rate-means table reports raw separation spread, not a standard deviation of the converted mean ψ. This nonlinear conversion differs from averaging per-scan ψ values.

surface_coverage_analysis

```
e.surface_coverage_analysis(cvs, options=None)
```

Estimates surface coverage Γ (mol/cm²) and total electroactive loading (mol) from the independent i_p-versus-scan-rate and tangent-corrected charge equations. Loading remains available without area; Γ requires area.

Equations

- $i_p = \frac{n^2 F^2 A \Gamma \ \ v}{4 R T}$
- $Q = n F A \Gamma ; \ \ \text{loading} = A \Gamma$

Automatic integration searches for tangent-baseline returns around each peak. Use integration range for an explicit potential window. Branch and method disagreements beyond agreement tolerance warn and retain both estimates rather than silently averaging them.

Parameters

Parameter	Meaning
cvs	A consistent surface-confined CV scan-rate series.
options	Segment and peak selection, n, electrode area, temperature, integration range and agreement tolerances.

Returns

AnalysisResult with per-CV estimates, summary, diagnostics, warnings and diagnostic figures. Loading can be estimated without area; coverage in mol/cm² requires area.

Example

```
coverage = e.surface_coverage_analysis(
    cvs,
    {
        "segments": [1, 2],
        "guess potential": [-0.1, -0.1],
        "integration range": "auto",
    },
)
```

Selected options

Option	Default	Meaning
agreement tolerance	0.25	Maximum symmetric fractional disagreement for branch and slope/charge surface-coverage comparisons; globally configurable.
integration range	'auto'	Potential window for tangent-corrected charge integration. Auto finds baseline returns around each selected peak; [minimum, maximum] sets an explicit window.
min r2	0.98	Minimum R2 for treating the ip-versus-scan-rate regression as reliable evidence.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.
num electrons	1	Number of electrons in the redox event.
peak prominence	None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
peak separation tolerance	0.01	Practical zero-separation tolerance in V used by surface reversibility diagnostics.
percent threshold	None	Percent threshold used in peak or tangent selection.
tangent min points	None	Minimum number of points used for tangent baseline fitting.
tangent potential	None	Potential at which to anchor tangent baseline fitting. In complex CV analyses, the plural alias 'tangent potentials' accepts per-CV values.
tangent range	'auto'	Potential range for tangent baseline fitting.

Complete registered options: e.describe_options("surface_coverage_analysis").

Equations and parameter definitions

$$|i_p| = m\,v + b\,; \quad \Gamma_{\text{slope}} = \frac{4\,R\,T\,m}{n^2\,F^2\,S}$$

$$Q = \frac{\left|\int \left[\, i(E) - i_{\text{baseline}}(E) \,\right] dE\right|}{v} \quad \text{(trapezoidal quadrature)}$$

$$\Gamma_{\text{charge}} = \frac{Q}{n\,F\,S}\,; \quad \text{loading} = S\,\Gamma = \frac{Q}{n\,F}$$

Symbol	Meaning and units	Input or controlling option
Γ	Surface coverage (mol/cm^2)	Derived separately from scan-rate slope and integrated charge.
m, b	Slope (A s/V) and intercept (A)	fit indices; min r2 reports fit quality.

Symbol	Meaning and units	Input or controlling option
n, S, T	Electron count, area (cm^2), temperature (K)	num electrons; electrode area / temperature or shared metadata.
v, i_baseline	Scan rate (V/s), tangent baseline (A)	CV scan_rate; tangent and peak-selection options.
Integration limits	Potential interval (V)	integration range; agreement tolerance compares branch and method estimates.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

Requires the reversible surface-confined model and a valid baseline. Q is the absolute net integral, not the integral of |i|. Coverage requires area; loading can remain available without area. Do not substitute bulk diffusion formulas.

Method and further reading

Read for the surface-confined model and its assumptions. The eCAT charge and scan-rate estimates require valid baseline subtraction and a justified electron count.

- Laviron, E. (1979). General expression of the linear potential sweep voltammogram in the case of diffusionless electrochemical systems. J. Electroanal. Chem. Interfacial Electrochem. 101, 19–28. https://doi.org/10.1016/S0022-0728(79)80075-3

plateau_current

```
e.plateau_current(cvs, options=None)
```

Analyzes scan-rate-independent catalytic plateau currents. Use e.plateau_current(cvs, options) for grouped scan-rate validation, or cv.plateau_current(options) as a single-CV convenience wrapper around the same workflow. When non-catalytic CVs are supplied with plot all=True, eCAT shows the reference/Sevcik-style diagnostic first, then catalytic plateau extraction, then plateau validation.

Parameters

Parameter	Meaning
cvs	catalytic CV or list of catalytic CVs.
options (dict or PlateauCurrentOptions)	peak-current extraction, non-catalytic reference, plateau-validation, formula, print, and plotting controls.
Peak-current extraction	plateau_current() now delegates no-peak fallback behavior to peak_current(). Use 'peak fallback' to choose 'highest current' (default), 'guess potential', or None for strict errors. For non-catalytic reference series, plateau_current() may still reuse a successful extraction potential from a nearby scan rate as an exact-potential hint.

Key options

non-catalytic cv / non-catalytic cvs, i_p^0, i_p^0 scan rate, i_p^0 sqrt scan rate slope, formula mode, validate plateau, require plateau, plateau average method, plot all

Returns

AnalysisResult with result.table for the plateau summary, result.summary for resolved kinetics/settings and result.diagnostics for extraction and validation details. Both grouped and single-condition paths use this contract.

Example

```
result = e.plateau_current(
    cvs,
    {
        "guess potential": -1.6,
        "non-catalytic cvs": blanks,
        "turnover electrons": 2,
        "plot all": True,
    },
)
```

Selected options

Option	Default	Meaning
background correction	'tangent'	Background correction method used before kinetic analysis. Choices: tangent, start current.
catalyst electrons	1	Catalyst redox-wave electron count n_cat used in kinetic equations. Aliases: n_cat, ncat.
diagnostic y axis	'i/ip0'	Y-axis for plot-all CV overlays. The default 'i/ip0' makes one combined normalized overlay for catalytic CVs and available non-catalytic CVs when ip0 can be resolved from manual inputs, non-catalytic CVs, or an ip0 sqrt scan-rate slope; otherwise plateau_current falls back to current and records a warning. Choices: i/ip0, current.
fit	True	Whether to fit the analysis result.
fit basis	'x'	Axis or quantity used to select the fit region. Choices: x, y.
ic	None	Manual catalytic plateau current alias for ilim. If omitted, plateau_current extracts it from catalytic CVs with peak_current.
ilim	None	Manual catalytic limiting current. If omitted, plateau_current extracts it from catalytic CVs with peak_current.
ip0 scan rate	None	Scan rate associated with manual ip0; if omitted, scan rate may supply it.
ip0 sqrt scan rate slope	None	Manual forced-origin slope of ip0 versus sqrt(scan rate); if omitted, it can be fitted from multiple non-catalytic CVs.
min fit points	20	Minimum recommended number of fit points.
min r2	0.98	Minimum recommended R2 threshold.
noise polyorder	'auto'	Savitzky-Golay smoothing polynomial order.

Complete registered options: e.describe_options("plateau_current").

Equations and parameter definitions

Direct: $k_{obs} = [\frac{|i_{lim}|}{nFSC}]^2 \frac{1}{Dn'}$

Reference peak: $k_{obs} = [0.446 | \frac{i_{lim}}{i_p^0} | \sqrt{\frac{nFv_0}{RT}}]^2 / n'$

Reference slope: $k_{obs} = [\,0.446 | \frac{i_{lim}}{s_0} | \sqrt{\frac{nF}{RT}}\,]^2 / n'$

Symbol	Meaning and units	Input or controlling option
i_{lim}	Catalytic plateau current (A)	ilim / ic or extracted wave; plateau selection mode / plateau average method.
i_p^0, v_0, s_0	Reference peak (A), its scan rate (V/s), slope of i_p^0 vs sqrt(v)	ip0; ip0 scan rate; ip0 sqrt scan rate slope; reference CV options.
n, n′	Catalyst and turnover electron counts	catalyst electrons / turnover electrons.
D, S, C	cm^2/s, cm^2, mol/cm^3	D; electrode area; C / c unit / species or metadata.
T, k_{obs}	Temperature (K); apparent rate (s^{-1})	temperature; formula mode selects direct, normalized or slope-normalized calculation.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

These are the implemented plateau expressions, including coefficient 0.446; they are not identical to the FOWA conversion. require plateau / validate plateau and plateau slope tolerance assess selection, not mechanistic validity. A custom formula replaces this conversion.

Method and further reading

Check that the plateau represents the applicable kinetic regime before converting current to a rate. Transport, depletion and deactivation can change the observed plateau.

- Rountree, E. S.; McCarthy, B. D.; Eisenhart, T. T.; Dempsey, J. L. (2014). Evaluation of Homogeneous Electrocatalysts by Cyclic Voltammetry. Inorg. Chem. 53, 9983–10002. https://doi.org/10.1021/ic500658x
- Costentin, C.; Drouet, S.; Robert, M.; Savéant, J.-M. (2012). Turnover Numbers, Turnover Frequencies, and Overpotential in Molecular Catalysis of Electrochemical Reactions. Cyclic Voltammetry and Preparative-Scale Electrolysis. J. Am. Chem. Soc. 134, 11235–11242. https://doi.org/10.1021/ja303560c

trumpet_analysis

```
e.trumpet_analysis(cvs, options=None)
```

Analyzes scan-rate-dependent peak splitting from paired CV segments and returns a ScatterFitResult, an AnalysisResult child. Peak selection and fit controls are routed through typed options.

Parameters

Parameter	Meaning
cvs	list of CVs.
options (dict or TrumpetAnalysisOptions)	paired-peak selection, fit indices and display controls. The x coordinates are log10(scan rate); there is no arbitrary x transform option.

Key options

Option	Default / type	Purpose and choices
segment	1 int or None	Base segment used to locate the paired peak branch. Cathodic/anodic identity is determined from each segment's potential scan direction, not from segment number or input order.
guess potential(s)	None float or list[float] or list[list[float]] or None	Initial potential guess for automatic peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
exact potential(s)	None float or list[float] or None	Exact potential for current extraction; when provided it bypasses peak-location auto-selection. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
fit indices	None object or None	Row/position-based fit selection. Use [start, stop] with Python-style exclusive stop, None for an open-ended side, a boolean mask, explicit indices, multiple windows for one disconnected fit, or a dict for named fits.
plot	True bool	Whether to plot the result.
plot all	False bool	Show a shared CV overlay with the actual paired peak selections used in the trumpet fit, in addition to the trumpet plot. Inspect these diagnostics before interpreting fitted kinetics.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
legend	False bool	Whether to show a plot legend.

Returns

ScatterFitResult with paired peak-potential data, fitted coefficients, diagnostics and optional figure/axes.

Example

```
result = e.trumpet_analysis(
    cvs, {"segments": [1, 2], "guess potential": -0.5}
)
```

Paired-branch limitation

trumpet_analysis currently resolves a pair of opposing segments. Passing segment=3 selects a base segment and triggers adjacent-segment pairing; it does not request a one-branch analysis. Both peaks must be resolved and scientifically assigned to the same couple.

- For one selected branch over multiple scan rates, use fit_peak_potential with explicit selection and x-axis settings to report a descriptive peak-shift trend. Do not interpret a missing return peak as a zero peak or infer both a formal potential and a rate constant from one fitted line.

Equations and parameter definitions

$$E_{pc} = m_c \log_{10}(v) + b_c;\ \ E_{pa} = m_a \log_{10}(v) + b_a$$

$$x_{\text{int}} = \frac{b_c - b_a}{m_a - m_c}$$

$$\alpha = -\frac{R\,T\ln(10)}{2\,F\,m_c};\quad \beta = \frac{R\,T\ln(10)}{2\,F\,m_a}$$

$$k^0 = \frac{10^{0.78 + x_{int}/2}}{\sqrt{\frac{R\,T}{\alpha\;\;F\,D}}}$$

Symbol	Meaning and units	Input or controlling option
m_c, m_a, b_c, b_a	Branch slopes (V/decade) and intercepts (V)	fit indices selects the fitted scan-rate interval; paired segments required.
v, x_{int}	Scan rate (V/s); branch intersection in log10(V/s)	CV scan_rate metadata.
D, T	Diffusion coefficient (cm^2/s), temperature (K)	D enables k0; temperature option or object metadata.
α, β, k^0	Derived transfer coefficients; rate (cm/s)	Returned by the implemented paired-branch asymptotic calculation.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

These are the current standalone formulas, including their fixed factor 2 and constant 0.78; there is no num electrons control here. This is not the surface Laviron calculation in reversibility_analysis, which uses ln(v) and returns s^{-1}. Treat standalone kinetic estimates as model-dependent; Laviron is background for the separate surface interpretation, not a citation validating these exact formulas.

Method and further reading

Laviron supplies background for surface-confined kinetics, used separately in reversibility_analysis. The standalone trumpet_analysis implements the diffusion-dependent formulas printed here; this citation does not establish the provenance of its fixed factor 2 and constant 0.78. Report branch trends descriptively unless the selected kinetic model is independently justified.

- Laviron, E. (1979). General expression of the linear potential sweep voltammogram in the case of diffusionless electrochemical systems. J. Electroanal. Chem. Interfacial Electrochem. 101, 19–28. https://doi.org/10.1016/S0022-0728(79)80075-3
- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

nicholson_analysis

```
e.nicholson_analysis(cvs, options=None)
```

Estimates heterogeneous electron-transfer kinetics from paired peak separations in a scan-rate series. Nicholson analysis requires a diffusion coefficient and is most appropriate when peak separation reflects electron-transfer kinetics rather than referencing, uncompensated resistance, adsorption, or background subtraction artifacts.

Parameters

Parameter	Meaning
cvs	CV or list of CVs containing the redox couple to analyze.

Parameter	Meaning
options (dict or NicholsonOptions)	segment selection, D, psi source, valid delta-E_p range, fit, print, and plotting controls. See e.describe_options("nicholson_analysis").

Key options

D, segment, num electrons, psi source, fit through origin, exclude invalid delta ep, plot all, plot diagnostic, warn ir drop

Returns

AnalysisResult with result["data"] and result.table for the Nicholson point table and result["summary"] / result.summary for equation metadata, fit statistics, and kinetic values such as k0. With print=True, eCAT prints the Nicholson equation and summary. The input/result table additionally requires print all=True. With plot all=True, eCAT makes one CV half-wave diagnostic plot and one Nicholson fit plot.

Example

```
result = e.nicholson_analysis(
    cvs,
    {
        "segment": 1,
        "guess potential": -0.5,
        "D": 1e-5,
        "plot all": True,
    },
)
result["summary"]
```

Selected options

Option	Default	Meaning
D	None	Diffusion coefficient in cm^2/s for heterogeneous rate reporting; not inferred.
psi source	'agarwal table'	Source for Nicholson psi values: Agarwal table or empirical Lavagnini equation. Choices: agarwal table, empirical.
num electrons	1	Number of electrons in the redox event.
fit model	'origin'	Nicholson fit model. 'origin' uses the theoretical ψ = k0 x relation; 'linear' adds a fitted intercept as a diagnostic backup. Choices: origin, linear.
nicholson delta ep min mv	61	Minimum valid nDelta Ep in mV for Nicholson analysis.
nicholson delta ep max mv	212	Maximum valid nDelta Ep in mV for Nicholson analysis.
exclude invalid delta ep	True	Exclude Nicholson points outside the valid nDelta Ep range from fitting.
fit through origin	True	Legacy Nicholson fit toggle. When both are provided, 'fit model' wins: 'origin' constrains through the origin and 'linear' fits an intercept.
noise polyorder	'auto'	'auto' chooses a Savitzky-Golay polynomial order compatible with the resolved smoothing window.
noise window	'auto'	'auto' chooses an odd Savitzky-Golay window from the selected data length; None disables Savitzky-Golay smoothing.

Option	Default	Meaning
peak prominence	None	Minimum peak prominence for automatic peak detection; None uses an automatic noise estimate, local to the guess potential when one is provided.
percent threshold	None	Percent threshold used in peak or tangent selection.

Complete registered options: e.describe_options("nicholson_analysis").

Choosing the shared conversion

```
shared = {"D": 1e-5, "psi source": "agarwal table"}
nicholson = e.nicholson_analysis(cvs, shared)
reversibility = e.reversibility_analysis(cvs, shared)
legacy = e.nicholson_analysis(cvs, {**shared, "psi source": "lavagnini"})
```

Equations and parameter definitions

$$z = 1000\, n\, |E_{pa} - E_{pc}| \ \text{(mV)}$$

Default: ψ = linear interpolation of Agarwal (2025), Table 4, at z.

$$\text{Optional Lavagnini: } \psi = \frac{-0.6288 + 0.0021\, z}{1 - 0.017\, z}$$

$$X = \sqrt{\frac{R\,T}{\pi\ \ D\, n\, F\, v}}$$

$$\psi = k^0\, X \ \text{(origin fit), or } \psi = k^0\, X + b \ \text{(linear fit)}$$

$$\text{Per-point } k^0 = \frac{\psi}{X}$$

Symbol	Meaning and units	Input or controlling option
ψ, z	Dimensionless kinetic parameter; scaled peak separation (mV)	psi source = agarwal table (default), or lavagnini / empirical.
E_{pa}, E_{pc}, n	Paired peak potentials (V), electron count	segment / segments and peak-selection options; num electrons.
D	Diffusion coefficient (cm^2/s)	D, required; one diffusion coefficient is used.
T, v	Temperature (K), scan rate (V/s)	CV temperature; fallback temperature when metadata absent. scan rate overrides CV metadata.
k^0, b	Heterogeneous rate (cm/s), optional dimensionless intercept	fit model = origin or linear. Default is origin.
zmin, zmax	Accepted peak-separation range (mV)	nicholson delta ep min mv / max mv (61 / 212); exclude invalid delta ep.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

The stored Agarwal table spans 61–214 mV; default rate-fit screening is 61–212 mV. Table interpolation returns NaN outside its domain, even if screening is relaxed. Lavagnini is an optional empirical approximation, not unrestricted

extrapolation. Assumes the applicable simple quasireversible diffusion-controlled couple; uncompensated resistance or coupled chemistry can invalidate the interpretation.

Method and further reading

The default ψ conversion interpolates Agarwal Table 4; psi source=lavagnini selects the Lavagnini approximation. Nicholson supplies the underlying model. Use paired peak separation only within the model and calibrated domain; diffusion coefficients, temperature and electron count are inputs, not inferred proof of the mechanism.

- Nicholson, R. S. (1965). Theory and Application of Cyclic Voltammetry for Measurement of Electrode Reaction Kinetics. Anal. Chem. 37, 1351–1355. https://doi.org/10.1021/ac60230a016
- Agarwal, R. (2025). The Nicholson Method of Determination of the Standard Rate Constant of a Quasireversible Redox Couple Employing Cyclic Voltammetry: Everything One Needs to Know! ACS Electrochemistry 1, 1885–1894. https://doi.org/10.1021/acselectrochem.5c00214
- Lavagnini, I.; Antiochia, R.; Magno, F. (2004). An Extended Method for the Practical Evaluation of the Standard Rate Constant from Cyclic Voltammetric Data. Electroanalysis 16, 505–506. https://doi.org/10.1002/elan.200302851
- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

scale_current

```
e.scale_current(cvs, options=None)
```

Returns copied CVs whose raw current columns are multiplied by a manual scale factor or by a factor resolved from reference-wave peak currents. reference mode="single" uses one selected peak; reference mode="both" uses explicit segments when provided, otherwise segment and segment+1, commonly i_p,c and i_p,a, to compute one signed scale factor. Original CV objects are not modified.

Parameters

Parameter	Meaning
cvs	list of CVs.
options (dict or ScaleCurrentOptions)	scale source, reference-CV, peak-current controls such as guess potential, print, and plot controls. See e.describe_options("scale_current").

Key options

Option	Default / type	Purpose and choices
reference index	0 int	Reference CV index used when scale/reference CVs are omitted; default 0 uses the first input CV.
scale	None float or list[float] or None	Manual current multiplier. If omitted, eCAT computes a scale factor from reference CV peak currents.
segment	1 int or None	CV segment to analyze.
segments	None list[int] or int or None	One or more CV segments to analyze.
guess potential	None float or None	Initial potential guess for peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.

Option	Default / type	Purpose and choices
exact potential	None float or None	Exact potential to use for peak or current extraction. In complex CV analyses, the plural alias 'exact potentials' accepts per-CV values.
tangent range	'auto' str or float or list[float] or tuple[float, float]	'auto' is passed to peak_current so reference-current extraction uses automatic tangent-baseline selection.
tangent potential	None float or None	Potential at which to anchor tangent baseline fitting. In complex CV analyses, the plural alias 'tangent potentials' accepts per-CV values.
print	False bool	Whether to emit output. False suppresses output; it is independent of pretty print.
plot all	False bool	Whether to show diagnostic or child plots.
reference mode	'single' str	'single' scales by one reference peak current; 'both' extracts paired segment currents and computes a best scale factor. Choices: single, both.

Returns

cv or list[cv], matching input shape, with scaled raw current data.

Example

```
scaled = e.scale_current(
    cvs, {"reference index": 0, "reference mode": "both", "segment": 1}
)
```

Equations and parameter definitions

$i_{\text{scaled}} = s\,i\,;$ reference mode chooses s to match the reference current(s)

Symbol	Meaning and units	Input or controlling option
s	Multiplicative scale factor	scale, or reference cv(s) / reference index / reference mode.

Scope and limitations

Scaling preserves current units but changes numerical current. It is distinct from a dimensionless i/i_p^0 column; retain scale provenance when interpreting quantitative rates.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

normalize_current

```
e.normalize_current(cvs, options=None)
```

Returns copied CVs with an i/i_p^0 column for FOWA-style normalized current plotting and analysis. Original CV objects are not modified.

Parameters

Parameter	Meaning
cvs	cv or list[cv].
options (dict or NormalizationOptions)	i_p^0 or reference CV source, peak-current controls and plot all/print options.

Key options

Option	Default / type	Purpose and choices
ip0	None float or list[float] or None	Manual non-catalytic reference peak current; if omitted, eCAT extracts ip0 from the selected reference CV.
reference cv	None object or None	Single reference CV used to extract ip0 automatically with peak_current.
reference cvs	None object or None	Reference CVs paired with input CVs to extract ip0 automatically with peak_current.
segment	None int or None	CV segment to analyze.
guess potential	None float or None	Initial potential guess for peak or wave selection. In complex CV analyses, the plural alias 'guess potentials' accepts per-CV values; scalar guesses keep running-guess behavior where supported.
tangent range	'auto' str or float or list[float] or tuple[float, float]	'auto' is passed to peak_current so ip0 extraction uses automatic tangent-baseline selection.
plot all	False bool	Whether to show diagnostic or child plots.
print	True bool	Whether to emit output. False suppresses output; it is independent of pretty print.
pretty print	True bool	Use rich table display when printing object lists or summaries. False uses plain-text output when print is True.

Returns

cv or list[cv], matching input shape.

Example

```
norm_cvs = e.normalize_current(
    cvs, {"reference cv": blank_cv, "segment": 1, "plot all": True}
)
```

Equations and parameter definitions

$$i_{\text{normalized}} = \frac{i}{i_p^0}$$

Symbol	Meaning and units	Input or controlling option
i, i_p^0	Current and reference peak current in matching units	ip0 or reference cv(s) / reference index; reference guess potential selects the reference wave.

Scope and limitations

The result is dimensionless. This wrapper copies CVs; the similarly named object method mutates its receiver. A reference peak must be nonzero.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

normalize

```
e.normalize(cvs, options=None)
```

Returns copied CVs with physical dimensionless potential and/or current axes for homogeneous or heterogeneous normalization. Original CV objects are not modified.

Parameters

Parameter	Meaning
cvs	cv or list[cv].
options (dict or NormalizeOptions)	physical constants and normalization mode. See e.describe_options("normalize").

Key options

- E^0 creates a stored column named "Dimensionless Potential".
- Current normalization creates a stored column named "Dimensionless Current" when D, concentration, electrode area, and scan rate are available; electrode area and scan rate may be collected automatically from the CV object.
- C and C unit may be provided explicitly. If they are omitted, species may be used to exact-match cv.compounds and pull the corresponding cv.concentrations entry; concentration is not guessed without species.
- Temperature defaults to cv.temperature whenever normalize creates a dimensionless axis. For current-axis normalization, electrode area defaults to cv.electrode_area and scan rate defaults to cv.scan_rate when not provided explicitly.
- D is expected in cm^2/s; concentration is converted internally to mol/cm^3; electrode area is cm^2; scan rate is V/s.

Option	Default / type	Purpose and choices
print	False bool	Whether to emit output. False suppresses output; it is independent of pretty print.
mode	'homogeneous' str	Normalization family: homogeneous uses Φ for current; heterogeneous uses χ. Choices: homogeneous, heterogeneous.
C	None float or str or None	Analyte or catalyst concentration. Explicit C overrides species-based lookup.
c unit	None str or None	Unit for numeric C. Required for numeric C; not needed when C is a concentration string or species resolves it.
D	None float or None	Diffusion coefficient in cm^2/s. Required for Dimensionless Current; not inferred.
E0	None float or None	Formal potential in V. Required for Dimensionless Potential; not inferred.

Option	Default / type	Purpose and choices
area	None float or None	Electrode area alias in cm^2. If omitted for Dimensionless Current, uses the CV's electrode_area.
temperature	None float or None	Temperature in K. If omitted, uses the CV's temperature.

Behavior notes

- The wrapper copies inputs; cv.normalize(...) mutates a single CV and returns self.
- Normalized CVs use the normalized axes by default for x(), y(), and plot() when those axes were created. Explicit x axis or y axis options can still select raw columns such as Potential or Current.
- For a series, normalize(..., {'print': True}) prints the symbolic normalization equation once, reports the shared mode outside the table, then shows a parameter table with parameters as rows and group-index columns. For one CV, the value column is labeled Value. Use 'pretty print': False for plain-text output.
- Default plot labels use compact symbols: θ for dimensionless potential, Φ for homogeneous dimensionless current, and χ for heterogeneous dimensionless current. The full equations are shown when normalize(..., {'print': True}) is used.
- normalize_current(...) is separate from physical dimensionless normalization. It creates i/i_p^0 from the raw Current column; it does not compute Phi/Phi_p0.

Returns

cv or list[cv], matching input shape.

Example

```
norm = e.normalize(
    cvs,
    {"mode": "homogeneous", "E0": 0.0, "D": 1e-5, "C": 10, "C unit": "mM"},
)
norm = e.normalize(
    cv, {"mode": "homogeneous", "E0": -1.0, "D": 1e-5, "species": "[Co]"}
)
```

Equations and parameter definitions

$$\theta = \frac{nF\left(E - E^0\right)}{RT}$$

$$\Phi \text{ or } \chi = \frac{i}{nFSC\sqrt{\frac{DnFv}{RT}}}$$

Optional homogeneous $\lambda = \frac{k_{\text{homo}} RT}{nFv}$

Optional heterogeneous $\psi = \frac{k^0}{\sqrt{\frac{\pi\ DnFv}{RT}}}$

Symbol	Meaning and units	Input or controlling option
θ, Φ, χ	Dimensionless potential and current	mode selects normalization presentation; E0 requests potential normalization, while current-related inputs request current normalization.

Symbol	Meaning and units	Input or controlling option
n, E^0	Electron count; reference potential (V)	num electrons / n; E0.
D, S, C	cm^2/s, cm^2, mol/cm^3	D; electrode area / area / s; C / c unit / species.
T, v	Temperature (K), scan rate (V/s)	temperature; scan rate / v; object metadata when not overridden.
k_{homo}, k^0	Homogeneous rate (s^{-1}); heterogeneous rate (cm/s)	k homo or k0 adds the corresponding dimensionless kinetic parameter in the selected mode.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

Use the centimeter-based unit combination shown here. Potential and current normalization do not validate a diffusion/kinetic model; homogeneous/heterogeneous modes use these same normalization scales.

Method and further reading

Background on peak selection, scan-rate dependence and interpretation. eCAT's baseline, smoothing and normalization options remain implementation choices; record them with the result.

- Elgrishi, N.; Rountree, K. J.; McCarthy, B. D.; Rountree, E. S.; Eisenhart, T. T.; Dempsey, J. L. (2018). A Practical Beginner's Guide to Cyclic Voltammetry. J. Chem. Educ. 95, 197–206. https://doi.org/10.1021/acs.jchemed.7b00361

tafel_analysis

```
e.tafel_analysis(cv, TOF_max, thermodynamic_potential, redox_potential, options=None)
```

Calculates and plots Tafel-style turnover-frequency curves for one catalytic CV or a CV series. Multi-CV calls use the same shared label, color, gradient, legend, and colorbar behavior as multiplot.

Parameters

Parameter	Meaning
cv_or_list	catalytic CV, or a list of catalytic CVs.
TOF_max	scalar value, or one value per CV. thermodynamic_potential and redox_potential set the overpotential relationship.
options (dict or TafelOptions)	Tafel controls plus common multiplot styling options such as labels, gradient by, legend mode, and colorbar tick labels.

Key options

Option	Default / type	Purpose and choices
overpotential range	[0, 1] list[float] or tuple[float, float]	Potential range used for Tafel or overpotential analysis.
color	'black' str	Primary plot color.

Returns

AnalysisResult with result["data"] / result.table for Tafel points, result["summary"] for the summary table, and result.axes for the plot axes.

Example

```
taf = e.tafel_analysis(
    cvs,
    TOF_values,
    E_thermo,
    E_redox,
    {"gradient by": "concentration", "legend mode": "colorbar"},
)
```

Equations and parameter definitions

$$\mathrm{TOF}(\eta) = \frac{2\,\mathrm{TOF}_{\max}}{1 + \exp\left[\frac{F\left(E_{\mathrm{thermo}} - E_{\mathrm{redox}} - \eta\right)}{R\,T}\right]}$$

$$y = \log_{10}(\mathrm{TOF})$$

Symbol	Meaning and units	Input or controlling option
TOF_{max}	Input rate (s^{-1})	TOF_max positional argument; scalar or one per CV.
E_{thermo}, E_{redox}, η	Thermodynamic potential, redox potential, overpotential (V)	thermodynamic_potential / redox_potential arguments; overpotential range option.
T	Temperature (K)	CV temperature metadata.
F, R	Faraday constant (C/mol); gas constant (J/mol/K)	Package constants; no user option.

Scope and limitations

This generates a molecular catalytic TOF curve; it does not fit a generic heterogeneous Tafel slope. The implemented expression approaches 2 × TOF_max at large positive η, despite the argument name. Do not interpret the input as the asymptotic value without checking the convention.

Method and further reading

These references concern molecular catalytic TOF–overpotential relationships. Do not interpret this function as a general heterogeneous-electrode Tafel-slope fitting routine.

- Costentin, C.; Drouet, S.; Robert, M.; Savéant, J.-M. (2012). Turnover Numbers, Turnover Frequencies, and Overpotential in Molecular Catalysis of Electrochemical Reactions. Cyclic Voltammetry and Preparative-Scale Electrolysis. J. Am. Chem. Soc. 134, 11235–11242. https://doi.org/10.1021/ja303560c
- Rountree, E. S.; McCarthy, B. D.; Eisenhart, T. T.; Dempsey, J. L. (2014). Evaluation of Homogeneous Electrocatalysts by Cyclic Voltammetry. Inorg. Chem. 53, 9983–10002. https://doi.org/10.1021/ic500658x

Simulation and fitting

Release scope

These optional workflows use standard ElectroKitty >=1.0.11.6. Install ecat-electrochemistry[simulation]. Importing ecat does not require the backend; executing a simulation does. The release does not include the experimental internal-grid

separation, staged fitting, repaired-backend certification or dynamic local-equilibrium adapter. Parameter checks are interpretation aids, not per-step solver or grid-convergence certification.

```
python -m pip install "ecat-electrochemistry[simulation]"
```

Simulation objects and ownership

Object	Contents and use
SimulatedCVInput	E and t arrays in V and s; optional measured current i in A, plus metadata. cv_program creates a waveform; cv_data selects measured data. quiet_time and incubation_time are metadata, not entries in E/t.
SimulatedCV	Simulated current in data with normal CV plotting/access. input_params retains normalized entered amounts, so changed-copy reruns do not reuse equilibrated concentrations.
SimulationFitResult	Single-curve fit parameters, statistics and measured/simulated plot diagnostics. This does not certify a unique mechanism or numerical convergence.
SimulationGroupFitResult	Shared and per-CV parameters, best_params_by_cv and group fit plots. per_cv identifies dataset-specific parameter paths.

cv_data reconstructs time from the selected potential sequence and scan rate; it does not preserve recorded Time values or explicit holds, even with stride=1. Selection and stride also determine the backend input grid in this release. Cropping can remove preceding waveform history; thinning can change numerical accuracy as well as cost. Retain physically important preceding sweeps and holds and check results at finer spacing before interpreting a fit. The separate observation/program/internal-grid design is not shipped here.

with_scan_rate() returns a changed copy using the release time-rescaling behavior; it does not offer the experimental hold-preserving waveform transformation. SimulatedCV.with_param(), with_params(), with_input(), with_mechanism() and with_incubation_time() rerun and return changed copies. Dictionary updates deep-merge; lists/scalars replace. Show methods print setup/parameters/states; input.plot() defaults to the waveform and fit-result.plot() overlays experiment and fit.

simulation.compile_mechanism

```
e.simulation.compile_mechanism(mechanism, params=None)
```

Compile preset names (E, EE, EC, ECE, EC' / Ecat, Square, and surface variants) or explicit E:/C: reaction strings. Returns a MechanismSpec dataclass. Coefficients such as C:A+2B=C are expanded for the backend; a leading integer cannot be part of a literal species name.

Parameters

Parameter	Meaning
mechanism	Preset name, reaction-string text or MechanismSpec; required. params optionally supplies species information.

simulation.cv_program

```
e.simulation.cv_program(Ei, E_low=None, E_high=None, Ef=None, scan_rate=0.1,
direction='negative', segments=2, points_per_segment=1000, quiet_time=0.0,
incubation_time=0.0)
```

Create SimulatedCVInput from vertices, scan direction, scan rate and point count. Ei and the initial-direction vertex are required: E_low for negative direction, E_high for positive direction. Other vertices are resolved from the entered values. segments counts sweeps. quiet_time and incubation_time are nonnegative seconds retained as metadata.

Parameters

Parameter	Default	Meaning
Ei	required	Initial potential / V
E_low, E_high, Ef	None	Vertices and final potential / V
scan_rate	0.1	V/s
direction	negative	Initial scan direction
segments	2	Number of sweeps
points_per_segment	1000	Samples per sweep
quiet_time, incubation_time	0.0	Hold and prior chemical incubation / s

simulation.cv_data

```
e.simulation.cv_data(cv, options=None)
```

Create fit-ready SimulatedCVInput from a measured cv after segment/window selection, optional background correction and stride. Returns selected E/i arrays and reconstructed t plus source metadata. start current subtracts the first selected current before thinning. Tangent correction uses the specified baseline controls.

Selected options

Option	Default / type	Purpose and choices
potential window	None list[float] or None	Potential window used to select measured CV data before simulation or fitting. With trim mode='expand', eCAT keeps connected scan data needed to preserve segment continuity.
segments	None int or list[int] or None	CV segment or segments to extract. If omitted, eCAT uses the selected CV data according to the window and trim mode.
stride	'auto' int or str	'auto' chooses a downsampling stride from points, points per volt, and min/max point targets; an integer keeps every nth selected point.
background correction	None str or bool or None	Measured-current background subtraction applied after potential-window/segment selection and before stride. 'start current' subtracts the first selected current point; 'tangent' subtracts a fitted tangent baseline. Choices: None, start current, tangent.
tangent potential	None float or None	Manual tangent anchor potential used by cv_data background correction when 'background correction' is 'tangent'.

Option	Default / type	Purpose and choices
tangent range	'auto' str or list[float] or tuple[float, float]	Potential range used only when background correction is 'tangent' and eCAT anchors the tangent from a peak/exact/guess potential.
estimate Cdl	'auto' bool or str	'auto' estimates total Cdl in F from measured current separation near the start potential when measured current and scan-rate metadata are available; False skips the estimate.

Complete registered options: e.describe_options("simulation.cv_data").

simulation.simulate_cv

```
e.simulation.simulate_cv(input, mechanism, params, options=None, backend='electrokitty')
```

Simulate one input with a mechanism and physical parameter mapping. Returns SimulatedCV. input must be SimulatedCVInput (use cv_data(cv) to convert a measured CV); mechanism and params are required. backend defaults to electrokitty.

Selected options

Option	Default / type	Purpose and choices
plot	True bool	Plot the simulated CV immediately after the backend simulation completes.
check params	False bool	Print diagnostic checks for likely simulation-parameter interpretation issues such as missing diffusion values or mechanism/species mismatches.
print params	False bool or str	Print prepared simulation parameters. True uses the default pretty table; 'compact' groups related parameters; 'raw' prints raw dictionaries.
print setup	False bool or str	Print simulation input/mechanism setup before or after simulation; use 'raw' for debugging-style output.
print states	False bool	Display entered, equilibrated, and post-incubation concentrations for species whose prepared amount changed.

Complete registered options: e.describe_options("simulation.simulate_cv").

simulation.fit_cv

```
e.simulation.fit_cv(input_or_result, mechanism=None, params=None, fit=None, options=None,
backend='electrokitty', method='least squares')
```

Fit measured current for one CV, fit-ready input or prior simulation. mechanism/params can be inherited from a result. fit specifies varying paths and bounds; method selects the optimizer, default least squares. Returns SimulationFitResult.

Selected options

Option	Default / type	Purpose and choices
cv data	None dict or None	Nested options passed to simulation.cv_data when fit_cv receives a real eCAT CV object.
max nfev	None int or None	Maximum optimizer function evaluations. Structured method dictionaries can override this budget.

Option	Default / type	Purpose and choices
print params	True bool or str	Print initial/final fit parameter tables with fit status and parameter paths.
print progress	False bool or str	Print the fitting progression table. Use 'all' to show every recorded evaluation; otherwise eCAT shows a compact summary. Choices: False, True, summary, all.
print setup	True bool or str	Print fitting setup, including method, residual, correction, budget, and varied/fixed parameter targets.
progress	'notebook' bool or str	Show fit progress. Notebook mode uses eCAT's progress display; False disables it.

Complete registered options: e.describe_options("simulation.fit_cv").

simulation.fit_cvs

```
e.simulation.fit_cvs(inputs_or_results, mechanism=None, params=None, fit=None, per_cv='auto',
options=None, backend='electrokitty', method='least squares')
```

Fit one shared mechanism across multiple input curves. per_cv marks dataset-specific paths; auto resolves supported dataset-specific values. fit and options use the single-fit conventions. Returns SimulationGroupFitResult.

Selected options

Option	Default / type	Purpose and choices
cv data	None dict or None	Nested options passed to simulation.cv_data when fit_cv receives a real eCAT CV object.
max nfev	None int or None	Maximum optimizer function evaluations. Structured method dictionaries can override this budget.
print params	True bool or str	Print initial/final fit parameter tables with fit status and parameter paths.
print progress	False bool or str	Print the fitting progression table. Use 'all' to show every recorded evaluation; otherwise eCAT shows a compact summary. Choices: False, True, summary, all.
print setup	True bool or str	Print fitting setup, including method, residual, correction, budget, and varied/fixed parameter targets.
progress	'notebook' bool or str	Show fit progress. Notebook mode uses eCAT's progress display; False disables it.

Complete registered options: e.describe_options("simulation.fit_cv").

Simulation parameter units and preparation

Simulation parameters use concentrations and diffusion for species setup:

```
params = {
    "concentrations": {
        "bulk": {"a": 1.0, "b": 0.0},
        "surface": {"cat*": 1e-9},
    },
    "diffusion": {"a": 1e-9, "b": 1e-9},
}
```

Simulation parameter values use SI-derived units unless a documented string alias or unit-bearing string is supplied:

Path or quantity	eCAT public unit	Notes
SimulatedCVInput.E, potential limits, kinetics.*.E0	V	Potentials are stored in volts.
SimulatedCVInput.t, incubation_time, quiet time	s	Incubation is chemical-only; quiet time is a backend hold at the starting potential.
simulated or measured current	A	Plotting may scale display units, but stored currents are amps.
scan_rate	V/s	Programmatic and CV-derived inputs use volts per second.
cell.T	K	"auto" pulls source CV metadata when available.
cell.Ru	Ω	Uncompensated resistance.
cell.Cdl	F	Total double-layer capacitance. cv_data(..., {"estimate Cdl": "auto"}) estimates total farads from current separation.
cell.A	m^2	Electrode area. Backend adapters that need areal capacitance compute cell.Cdl / cell.A internally.
concentrations.bulk.*	mol/m^3	Use 1000 mol/m^3 for 1 M; entered values are pre-equilibration amounts.
concentrations.surface.*	mol/m^2	Surface amounts are coverages.
diffusion.*	m^2/s	Applies to mobile bulk species.
kinetics.*.k0	m/s	String presets such as "fast", "quasi", and "slow" resolve to SI values.
kinetics.*.alpha	dimensionless	Charge-transfer coefficient.
spatial.dx_fraction	dimensionless	Spatial mesh fraction.
spatial.nx	count	Spatial mesh size setting.
spatial.viscosity	m^2/s	Kinematic viscosity; solvent aliases resolve to approximate room-temperature values.
spatial.rotation	Hz	Rotating-disk frequency when used.

Path or quantity	eCAT public unit	Notes
activity.standard_concentration	mol/m³	Defaults to 1000 mol/m³, i.e. 1 M.
activity.standard_coverage	mol/m²	Surface-activity standard; defaults to 1 mol/m².
activity.gamma.*	dimensionless	Activity coefficients; omitted values behave as 1.
reactions.*.K	dimensionless by default	Dimensionless K is activity-based. Unit-bearing strings such as "2 M^-1" or "1e-3 m^3/mol" are converted using the activity standard concentration and gammas.
reactions.*.k, kf, kb	reaction-order dependent	First-order rates are s^{-1}; second-order rates are $m^3\ mol^{-1}\ s^{-1}$; third-order rates are $m^6\ mol^{-2}\ s^{-1}$, using the concentration units above.
reactions.*.k_exchange, koff	s^{-1}	Exchange or reverse rate scales used with K before compiling backend kf/kb.

kinetics and reactions are user-facing mechanism parameter sections. They may be lists, integer-keyed dictionaries, or dictionaries keyed by mechanism reaction strings. kinetics entries describe electrochemical E: steps with E0, k0, and alpha. reactions entries describe chemical C: steps and may use irreversible k, reversible kf/kb, or equilibrium-derived K plus k_exchange or koff. A string K may include units, such as "2 M^-1" or "1e-3 m^3/mol". eCAT compiles these physical inputs into private backend-ready rates under params["_compiled"]. Fitting can still vary physical paths such as reactions.0.K and reactions.0.k_exchange; for each trial simulation, eCAT converts the current physical values into the kf/kb rates required by the backend.

Equilibrium units, updates and fitting

A bare numeric K is dimensionless and uses activities. A string such as "2 M^-1", or {"K": 2, "K_unit": "M^-1"}, supplies a concentration quotient. For products-minus-reactants stoichiometric numbers ν_i, define p = n_r - n_p and Γ = product(gamma_iv_i). Then:

$$K = K_{c,native} \cdot c_{standard}{}^{p} \cdot \Gamma$$

$$K_{c,native} = \frac{K}{c_{standard}{}^{p} \cdot \Gamma}$$

Symbol	Definition	Input/control
K	Dimensionless activity equilibrium constant	reactions.*.K without concentration units
Kc_native	Product/reactant concentration quotient, $(mol/m^3)^{(-p)}$	Unit-bearing K; compiled kf/kb
c_standard	Bulk activity standard, mol/m^3	activity.standard_concentration; default 1000
gamma_i	Positive dimensionless activity coefficient	activity.gamma.bulk; default 1
n_r, n_p, ν_i	Total reaction orders and signed species coefficients	Chemical reaction stoichiometry

For ideal A+B=AB, 2 M^-1 = 0.002 m^3/mol: dimensionless K is 2 at 1000 mol/m^3 standard concentration and 0.2 at 100 mol/m^3. Both represent the same concentration quotient. The activity-based convention follows the IUPAC equilibrium-constant definition (https://goldbook.iupac.org/terms/view/E02177).

Supported unit-bearing inputs remain bulk association quotients: M^-1, mM^-1, m^3/mol and their existing aliases require p=1; explicit native uses the positive association order in the reaction. Unsupported powers, dissociation or surface concentration-unit inputs raise an error; use a correctly converted dimensionless activity K for those cases. Surface dimensionless constants use activity.standard_coverage. Structured {"kind": ...} inputs are not part of this release. Conflicting unit annotations and nonpositive or nonfinite constants are rejected.

For eCAT optimizers, the base equilibrium values, fit.fixed, fit.init and fit.bounds are normalized to dimensionless K once, using the entered parameter context after fixed standard-state and gamma updates. Mapping insertion order does not affect that conversion. Bare numbers in these fields are dimensionless; strings can carry the supported concentration units. best_params and fitted parameter tables use dimensionless values with K_unit="dimensionless". Simulation diagnostics include K_dimensionless, logK (natural logarithm), K_concentration, kf and kb.

Activity-coefficient fitting remains supported. Its optimizer starting values and trial changes do not redefine this fixed unit-conversion context. If gamma or the standard state is varied, fixed dimensionless K bounds do not remain fixed concentration-quotient bounds. Prefer explicit dimensionless K inputs and bounds for such joint fits; inspect the reported quotient for interpretation. Ordinary simulate_cv() preserves the entered unit convention in its parameter mapping while compiling consistent physical rates.

Group fits convert per-CV K starting values and bounds using each dataset's context. A shared K must have compatible dimensionless starts/bounds across curves; use explicit dimensionless starts/bounds or per_cv=["reactions.0.K"] when concentration-unit inputs would imply different shared settings. Other shared parameter bounds retain their existing behavior.

Replacing reactions.0.K through with_param() or with_params(set=...) replaces its unit convention: a bare number is dimensionless; a unit-bearing string is converted in the updated context. Simultaneous fixed/context changes are applied before K conversion. Deep-merged reaction mappings likewise discard old unit labels unless the update explicitly supplies them; lists still replace as a whole. These rules do not reinterpret chemical rate k or species named K.

Reaction-string dictionary keys may use either coefficient or repeated-species notation; for example, a "A+A=B" key matches a C:2A=B mechanism step. For a reaction with total reactant order n_r, product order n_p, and the phase-appropriate activity standard X°, eCAT defines the pool-free standard-state exchange scale as:

k_exchange = kf * X°**(n_r - 1) + kb * X°**(n_p - 1)

X° is activity.standard_concentration for bulk reactions and activity.standard_coverage for surface reactions. For first-order equilibria this reduces to kf + kb; for bulk A + B = C it is kf C° + kb. The activity-derived kf/kb ratio and this equation uniquely determine the backend rates. Those intrinsic rates do not change across a concentration series, while the actual mass-action rate still changes with the current concentrations. reference_concentration or reference_coverage may replace the corresponding phase standard for this rate-scale definition; the reference must be positive and finite.

Every species in an explicit-K reaction must have an entered concentration; use zero for an initially absent species. By default eCAT solves the complete pre-equilibrium from the reaction stoichiometric matrix, dimensionless activity quotients, and conservation laws inferred from that matrix. No pool declaration is required. A reversible reaction with equilibrate=False is excluded from the algebraic starting-state solve but still participates in finite incubation and backend dynamics. Consistent dependent equilibrium cycles are accepted; inconsistent cycles raise a residual-based error.

The preparation order is:

1. Normalize entered concentrations and physical reaction parameters.
2. Solve explicit-K reactions unless equilibrate=False.
3. Integrate bulk homogeneous chemical reactions for incubation_time when it is greater than zero. Surface and mixed-phase steps remain backend-only.
4. Apply quiet time as an ElectroKitty hold at the starting potential.
5. Run the CV waveform.

concentrations.pools, top-level pools, and top-level equilibria are not supported. Put K directly in the matching reactions entry. Pre-equilibrium currently requires every participating reaction to remain within one phase; mixed bulk/surface equilibria raise an error because eCAT has no volume-to-area capacity convention for a shared conservation equation.

eCAT reports numerical reaction rank, inferred conservation rank, dependent reaction count, and equilibrium residuals. It does not claim elemental or charge balance from species labels because names such as Substrate or coordination-complex abbreviations are not reliable molecular formulas.

Activity coefficients are activity["gamma"] values, default to 1, and are displayed inside the species table only when at least one gamma differs from 1:

```
params["concentrations"] = {"bulk": {"A": 10.0, "B": 0.0}}
params["reactions"] = {"A=B": {"K": 4.0, "k_exchange": 10.0}}
params["activity"] = {"gamma": {"bulk": {"A": 0.9}}}
```

For cell constants, use an explicit mapping when teaching or auditing values, or use "cell": "auto" once the workflow is established. The shorthand expands to {"Cdl": "auto"} and then fills T, Ru, and A from the source CV when available, falling back to 298.15 K, 0 Ω, and 1e-5 m². Cdl requires measured current or a prior cv_data(..., {"estimate Cdl": "auto"}) estimate. In eCAT's public params, cell.Cdl is total double-layer capacitance in F. The ElectroKitty adapter converts that total value to the backend's area-normalized capacitance internally using cell.Cdl / cell.A; users should not divide by area themselves or pass F/m² as cell.Cdl.

As input sugar, species may be supplied as a mapping of species names to type/C/D fields; eCAT normalizes it immediately into concentrations and diffusion, and prepared simulation results do not retain species. Use {"print params": "compact"} for compact species and mechanism parameter tables; fitting comparison tables remain path-rich.

Examples

```
program = e.simulation.cv_program(
    Ei=0.3, E_low=-0.4, E_high=0.3, scan_rate=0.1,
    points_per_segment=1000, quiet_time=0.0,
)
program.show()
program.plot()
# For measured curves, inspect the selected waveform before fitting:
# fit_input = e.simulation.cv_data(cvs[0], {"stride": 1})
# result = e.simulation.simulate_cv(fit_input, "E", params)
# fit_result = e.simulation.fit_cv(
#     fit_input, "E", params, fit={"vary": ["E0_0"], "bounds": "auto"}
# )
```

The example creates a program without running the optional backend. Use tutorials 10–12 for complete parameter dictionaries and group/equilibrium-derived fitting examples. Optimizer termination does not establish scientific agreement or discretization accuracy. Native ElectroKitty fitting and eCAT optimizers have different controls; use the actual installed option table and backend documentation.

Defaults and Option Discovery

Defaults precedence: package defaults < loaded TOML defaults < session defaults < per-call options. echem objects and analysis DataFrames expose unit metadata through .attrs['units'] where available; raw echem data still also use object.units for column units.

For describe_options itself, print controls whether anything is emitted, while pretty print only controls output style: {'pretty print': False} prints a plain text table, and {'print': False} suppresses output.

Option names accept spaces, underscores, and hyphens in most notebook-facing APIs, and registered choice values are case-insensitive. For choice-like options, spaces, underscores, and hyphens are also treated interchangeably. If an option explicitly lists 'none' as a valid choice, Python None, 'None', 'off', 'false', 'no', and '0' are interpreted as the canonical string 'none'; omitting the option still uses the default.

describe_options('function_name') reports function-specific option metadata: current defaults from eCAT's defaults system, dataclass-derived types, and function-specific choices and descriptions. This matters for shared option names such as mode, which can mean include/exclude for filter but homogeneous/heterogeneous for normalize.

When an option is resolved automatically, the Description column distinguishes the source. Metadata retrieval is described explicitly, such as using a CV's stored scan_rate, temperature, electrode_area, compounds, or concentrations.

- Algorithmic auto behavior is described as automatic selection from the data or plot context, such as unit scaling, tangent-region selection, gradient-color grouping, reference-wave detection, or result-column selection. Scientific inputs that are not inferred, such as some diffusion coefficients or formal potentials, are marked that way in the function-specific table.

Python docstrings now use concise NumPy-style summaries and point to e.describe_options('function_name') for current option tables, so option details stay in one place. Calling e.describe_options() by itself shows the valid option sections menu, starting with 'all'.

- Use e.describe_options('all') for every option row, e.describe_options('multiplot') for one section, or simulation tables such as e.describe_options('simulation.fit_cv'). Unknown section names print a friendly suggestion when available and show the same section menu.
- The helper displays tables by default; pass {'pretty print': False} for plain text output, {'print': False} to suppress output, and {'return': True} when you want the pandas DataFrame returned.

```
e.describe_options()
e.describe_options("fit_peak_current")
schema = e.describe_options("plot", {"print": False, "return": True})
e.set_defaults("fowa", {"fit basis": "y", "min r2": 0.98})
e.reset_defaults("fowa")
```

Canonical option-section names

Public function / section	Use with describe_options(...)
get_data	e.describe_options('get_data')
plot	e.describe_options('plot')
multiplot	e.describe_options('multiplot')
multimultiplot	e.describe_options('multimultiplot')
multi_scatterplot	e.describe_options('multi_scatterplot')
filter	e.describe_options('filter')
sort_group	e.describe_options('sort_group')
peak_current	e.describe_options('peak_current')
fit_peak_current	e.describe_options('fit_peak_current')
fit_peak_potential	e.describe_options('fit_peak_potential')
fit_rate	e.describe_options('fit_rate')
sevcik_analysis	e.describe_options('sevcik_analysis')
trumpet_analysis	e.describe_options('trumpet_analysis')
fowa	e.describe_options('fowa')

Public function / section	Use with describe_options(...)
normalize_current	e.describe_options('normalize_current')
scale_current	e.describe_options('scale_current')
tafel_analysis	e.describe_options('tafel_analysis')
normalize	e.describe_options('normalize')

describe_options

```
e.describe_options(option_model_or_section=None, options=None)
```

Show the registered option menu or one option table.

Parameters

Parameter	Meaning and default
option_model_or_section	None for the menu, a function/section name, "all", or a public option dataclass.
options	print, pretty print and return display controls.

Returns

None by default; pandas.DataFrame only when return=True. An invalid section displays/returns the menu, so verify its columns before treating it as an option table.

Example

```
schema = e.describe_options(
    "fit_peak_current", {"print": False, "return": True}
)
```

Selected options

Option	Default	Meaning
pretty print	True	Whether emitted output uses rich notebook display. False prints a plain text table; it does not suppress output.
print	True	Whether to emit the options table or menu. False suppresses output but still allows dataframe return.
return	False	Whether to return the options table/menu as a pandas DataFrame in addition to any printed output.

Complete registered options: e.describe_options("describe_options").

get_defaults

```
e.get_defaults(section=None)
```

Read merged package, user-file and session defaults.

Parameters

Parameter	Meaning and default
section	None for all sections or a section name.

Returns

Dictionary. Unknown sections currently return an empty dictionary.

Example

```
defaults = e.get_defaults("plot")
```

set_defaults

```
e.set_defaults(section_or_mapping, updates=None)
```

Set session defaults without writing a configuration file.

Parameters

Parameter	Meaning and default
section_or_mapping	Section name, global option name or nested mapping.
updates	Section dictionary or scalar value for global shorthand.

Returns

Dictionary of merged defaults after the change.

Example

```
e.set_defaults("plot", {"legend": False})
```

reset_defaults

```
e.reset_defaults(section=None)
```

Clear loaded/session overrides.

Parameters

Parameter	Meaning and default
section	None for everything; a section or global option name for a narrower reset.

Returns

Dictionary of resolved defaults. Existing objects are not reimported.

Example

```
e.reset_defaults("plot")
```

load_defaults

```
e.load_defaults(path)
```

Load user defaults from a TOML file into this session.

Parameters

Parameter	Meaning and default
path	Path to an existing TOML file.

Returns

Dictionary of merged defaults. This replaces the previously loaded user-file layer; it does not save a file.

Example

```
e.load_defaults("lab_ecat_defaults.toml")
```

Typed options and error classes

Public option dataclasses are convenience alternatives to dictionaries for workflows that accept them. Discover their fields with e.describe_options(e.PeakCurrentOptions). Dictionary aliases use spaces, underscores and hyphens where registered; do not assume every untyped helper accepts every alias. OptionError reports invalid validated options. UnsupportedFileFormatError identifies unsupported imports.

Public option classes	Workflow
CVFilterOptions, ImportOptions, TrimOptions, FilterOptions, SortGroupOptions, GroupSummaryOptions	Import, selection and collections
PlotOptions, SpectrumOptions, MultiplotOptions, MultiMultiplotOptions, MultiScatterplotOptions	Plotting and spectra
PeakPotentialOptions, PeakCurrentOptions, PeakWidthOptions, NormalizeOptions, NormalizationOptions, ScaleCurrentOptions, FOWAOptions, PlateauCurrentOptions, FitRateOptions, FitPeakPotentialOptions, SevcikAnalysisOptions, ReversibilityAnalysisOptions, SurfaceCoverageAnalysisOptions, FitPeakCurrentOptions, TrumpetAnalysisOptions, NicholsonOptions, TafelAnalysisOptions	Peak metrics, normalization and advanced analyses